

Rapport de Projet de Fin d'Années
Pour l'obtention du Diplôme de l'ESISA

BAC+3(*Filière* : Ingénierie logicielle)

Sous le thème

Développement d'une plateforme Web/PWA pour la vente de
produits agricoles frais (légumes, fruits, herbes)

Réalisé par :

BOURKKADI IDRISSI TAHA

ZMAROU Hamza

SGHIRI Salah Eddine

Encadré par :

Mr. BOULEALAM Chafik

Mr. IRAQI HOUSSAINI Mehdi

Membre du Jury :

Mr.

DÉDICACES

À mes chers parents,

Pour leur amour inconditionnel, leur soutien sans faille et leurs sacrifices silencieux qui m'ont permis d'arriver jusqu'ici. Aucun mot ne saurait exprimer ma gratitude envers vous. Ce travail est le fruit de votre confiance et de vos encouragements constants. Que ce modeste rapport soit le témoignage de mon profond respect et de mon amour éternel.

À mes amis et coéquipiers de l'équipe Souki,

Pour ces nuits de travail partagées, ces idées échangées, ces doutes surmontés ensemble et cette énergie qui n'a jamais faibli. Ce projet n'aurait jamais vu le jour sans votre engagement, votre créativité et votre esprit d'équipe. Merci d'avoir cru en cette vision et de l'avoir construite avec moi, ligne de code après ligne de code.

À tous ceux qui, de près ou de loin, ont contribué à la réussite de ce projet.

REMERCIEMENTS

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet.

Nos Encadrants de projets :

Nos premiers remerciements s'adressent à nos encadrants de projet, **IRAQI HOUSSAINI Mehdi** et **BOULEALAM Chafik** pour leur disponibilité, leurs conseils avisés et leur accompagnement précieux tout au long de ce travail. Leur expertise et leurs retours constructifs ont été déterminants dans la réussite de ce projet.

Membres de Jury :

Nous adressons également nos sincères remerciements aux membres du jury, pour l'intérêt qu'ils portent à ce travail ainsi que pour le temps consacré à son évaluation.

Mon institution :

Nos remerciements vont aussi à notre institution, l'ESISA, Ecole Supérieure d'Ingénierie en Sciences Appliquées, pour la qualité de la formation dispensée et l'excellence de son corps professoral, dont l'exigence et le savoir-faire nous ont permis d'acquérir les compétences nécessaires à la conception de ce projet. Nous tenons en particulier à remercier **Monsieur MEKOUAR Khalid**, Président, Fondateur et Directeur de l'ESISA, pour sa vision et son engagement constant en faveur de l'excellence académique et de l'innovation, qui font de l'ESISA un environnement propice à la réussite de ses étudiants.

Enfin, nous remercions toutes les personnes qui, par leur soutien, leurs encouragements ou leur disponibilité, ont contribué à la réalisation de ce projet.

RÉSUMÉ

Ce rapport présente le développement de SOUKI, une plateforme web innovante facilitant l'achat de fruits et légumes frais en circuit court au Maroc. Face au compromis imposé aux consommateurs

urbains entre les souks traditionnels chronophages et les applications de livraison généralistes aux marges élevées, notre solution propose une approche moderne, économique et adaptée au contexte marocain.

Le projet repose sur une architecture full-stack composée d'une API FastAPI/Python, d'une interface web Next.js/React et d'une application mobile React Native/Expo. Le code met en œuvre l'authentification JWT, la vérification OTP, la connexion Google, la gestion multi-rôles, le catalogue, le panier, le checkout, l'agrégation Just-In-Time, le dispatch logistique, le suivi des livraisons, l'espace fournisseur, les réclamations et SoukiWallet. La commande intelligente combine Gemini pour l'analyse texte/audio et un modèle Gemma-2 distant avec un jeu de compositions local comme mécanisme de repli.

L'analyse du dépôt confirme la cohérence technique de cette architecture et l'existence de tests automatisés ciblant notamment le JIT, la logistique, le cloisonnement et les fournisseurs. Les résultats financiers et les performances en production ne disposant pas encore de mesures auditable dans le dépôt, ils sont présentés comme des hypothèses de travail nécessitant une validation expérimentale.

Mots-clés : AgriTech, Circuit court, Application web, FastAPI, Next.js, Intelligence Artificielle, Maroc

ABSTRACT

This report presents the development of SOUKI, an innovative web platform designed to facilitate the purchase of fresh fruits and vegetables through a short supply chain model in Morocco. In response to the dilemma faced by urban consumers between time-consuming traditional markets and expensive general delivery applications, our solution offers a modern, affordable, and locally adapted approach.

The project uses a full-stack architecture composed of a FastAPI/Python API, a Next.js/React web interface, and a React Native/Expo mobile application. The codebase implements JWT authentication, OTP verification, Google sign-in, role-based access, catalogue and cart management, checkout, Just-In-Time aggregation, logistics dispatch, delivery tracking, supplier workflows, claims, and SoukiWallet. Intelligent ordering uses Gemini for text/audio analysis and a remote Gemma-2 model with a local dataset fallback.

Repository analysis confirms the technical consistency of the architecture and the presence of automated tests for JIT, logistics, access isolation, and supplier features. Because no auditable production measurements or validated financial results are stored in the repository, financial figures and performance targets are reported as assumptions that require field validation.

Keywords: AgriTech, Short Supply Chain, Web Application, FastAPI, Next.js, Artificial Intelligence, Morocco

الملخص

يقدم هذا التقرير تطوير يقدم هذا التقرير تطوير منصة SOUKI، وهي منصة ويب مبتكرة تهدف إلى تسهيل شراء الفواكه والخضروات الطازجة عبر نموذج السلسلة القصيرة للتوريد في المغرب.

انطلاقًا من التحدي الذي يواجه المستهلكين في المدن بين الأسواق التقليدية التي تتطلب وقتًا وجهدًا كبيرين وتطبيقات التوصيل العامة ذات التكاليف المرتفعة، تقترح منصتنا حلًا عصريًا واقتصاديًا ومتكيفًا مع الواقع المغربي.

يعتمد المشروع على بنية تقنية حديثة من نوع Full-Stack، حيث تم استخدام FastAPI و Python لتطوير الواجهة الخلفية، و Next.js و React لتطوير الواجهة الأمامية.

تشمل الوظائف المطورة نظام مصادقة آمن (JWT، OTP، Google OAuth)، وسلة مشتريات ذكية مدعومة بمساعد صوتي يعتمد على نموذج Gemma-2 المخصص، ونظام تجميع Just-In-Time للحد من الهدر الغذائي، بالإضافة إلى وحدة لإدارة التوزيع اللوجستي وتحسين مسارات التوصيل.

كما يضمن دمج المحفظة الرقمية SoukiWallet ووسائل الدفع المتعددة (الدفع نقدًا وبطاقات CMI البنكية) تجربة استخدام مرنة وآمنة.

تؤكد النتائج جدوى الحل من الناحيتين التقنية والاقتصادية، حيث يتيح نموذج "صفر هدر" تحقيق تدفق نقدي صحي منذ الطلبات الأولى والوصول إلى نقطة التعادل ابتداءً من سبع طلبات يوميًا. تتموقع منصة SOUKI كحل رقمي متاح يساهم في تطوير سلسلة توريد المنتجات الطازجة بما يتناسب مع خصوصيات السوق المغربي.

الكلمات المفتاحية: التكنولوجيا الزراعية، السلسلة القصيرة للتوريد، تطبيق ويب، FastAPI، Next.js، الذكاء الاصطناعي، المغرب



المشروع يهدف إلى تطوير منصة رقمية متكاملة لتسهيل عمليات الدفع والتوريد في القطاع الزراعي. سيتم دمج المحفظة الرقمية SoukiWallet مع وسائل الدفع المتعددة (نقدًا وبطاقات CMI) لضمان تجربة استخدام مرنة وآمنة للمستخدمين. سيتم تحقيق تدفق نقدي صحي منذ الطلبات الأولى والوصول إلى نقطة التعادل ابتداءً من سبع طلبات يوميًا. تتموقع منصة SOUKI كحل رقمي متاح يساهم في تطوير سلسلة توريد المنتجات الطازجة بما يتناسب مع خصوصيات السوق المغربي.

تتموقع منصة SOUKI كحل رقمي متاح يساهم في تطوير سلسلة توريد المنتجات الطازجة بما يتناسب مع خصوصيات السوق المغربي. سيتم دمج المحفظة الرقمية SoukiWallet مع وسائل الدفع المتعددة (نقدًا وبطاقات CMI) لضمان تجربة استخدام مرنة وآمنة للمستخدمين. سيتم تحقيق تدفق نقدي صحي منذ الطلبات الأولى والوصول إلى نقطة التعادل ابتداءً من سبع طلبات يوميًا. تتموقع منصة SOUKI كحل رقمي متاح يساهم في تطوير سلسلة توريد المنتجات الطازجة بما يتناسب مع خصوصيات السوق المغربي.

تتموقع منصة SOUKI كحل رقمي متاح يساهم في تطوير سلسلة توريد المنتجات الطازجة بما يتناسب مع خصوصيات السوق المغربي. سيتم دمج المحفظة الرقمية SoukiWallet مع وسائل الدفع المتعددة (نقدًا وبطاقات CMI) لضمان تجربة استخدام مرنة وآمنة للمستخدمين. سيتم تحقيق تدفق نقدي صحي منذ الطلبات الأولى والوصول إلى نقطة التعادل ابتداءً من سبع طلبات يوميًا. تتموقع منصة SOUKI كحل رقمي متاح يساهم في تطوير سلسلة توريد المنتجات الطازجة بما يتناسب مع خصوصيات السوق المغربي.

Table De Matière

Résumé

Abstract

الملخص

。又々々々

Liste des abréviations

Liste des figures

Liste des tableaux

Introduction Générale

Chapitre 1 : Contexte et Problématique

- 1.1 Présentation du secteur agroalimentaire au Maroc
- 1.2 Problématique : le compromis frais / prix / temps
- 1.3 Solution proposée : SOUKI

1.4 Objectifs du projet

1.4.1 Objectifs techniques

1.4.2 Objectifs fonctionnels et business

Chapitre 2 : Analyse et Conception

2.1 Analyse des besoins fonctionnels

2.1.1 Identification des acteurs (Client, Parent, Livreur, Fournisseur, Admin)

2.1.2 Scenarios d'usage type

2.1.3 Cas d'utilisation détaillés

2.1.4 Exigences non-fonctionnelles

2.2 Etat de l'art / Analyse comparative du marché

2.2.1 Applications de livraison généralistes (Glovo, Kaalix, Khodrati)

2.2.2 Plateformes E-commerce bio (Green Village)

2.2.3 Marchés traditionnels physiques (Souks)

2.2.4 Notre différenciation (valeur ajoutée)

2.3 Analyse des risques du projet

2.4 Spécifications techniques

2.4.1 Vue d'ensemble de l'architecture

2.4.2 Architecture fonctionnelle (modules métier)

2.4.3 Architecture technique (Backend / Frontend / Mobile)

2.5 Architecture globale du système

2.5.1 Workflows détaillés (agrégation JIT, commande vocale, dispatch)

2.5.2 Gestion des données

2.5.3 Intégrations externes

2.5.4 Sécurité et authentification

2.6 Conception de la base de données (modèles PostgreSQL)

2.7 Diagrammes UML (Cas d'utilisation, Séquences)

2.7.1 Authentification classique (Email/Téléphone + OTP)

2.7.2 S'authentifier via OAuth (Google)

2.7.3 Créer une commande vocale

2.7.4 Processus d'agrégation JIT

2.8 Maquettes et prototypes de l'interface

2.8.1 Parcours Client

2.8.2 Parcours Livreur

2.8.3 Parcours Fournisseur

2.8.4 Parcours Administrateur

Chapitre 3 : Choix Technologiques et Architecture Technique

3.1 Architecture Full-Stack

3.1.1 Architecture Backend (FastAPI/Python)

3.1.2 Architecture Frontend (Next.js/React)

3.1.3 Architecture Mobile (PWA)

3.2 Justification des choix technologiques

3.2.1 Stack Backend (FastAPI, PostgreSQL, SQLAlchemy)

3.2.2 Stack Frontend (Next.js, TypeScript, Shadcn/UI, TailwindCSS)

3.2.3 Services tiers (Hugging Face, Gemini AI, Google OAuth, Supabase, PayZone)

3.3 Sécurité et authentification

Chapitre 4 : Réalisation et Implémentation

4.1 Modules développés

4.1.1 Module d'authentification et gestion multi-rôles

4.1.2 Module Catalogue et gestion des produits

4.1.3 Module Panier (manuel, texte, vocal)

4.1.4 Module IA - Assistant vocal (Gemma-2-2B fine-tune QLoRA)

4.1.5 Module Checkout et paiement

4.1.6 Module Agrégation JIT

4.1.7 Module Dispatch et optimisation des tournées

4.1.8 Module Livreur (tracking, événements de livraison)

4.1.9 Module Fournisseur (marketplace B2B)

4.2 Dashboards spécialisés

4.3 Intégrations externes

Chapitre 5 : Etude Financière et Modèle Economique

5.1 Modèle Lean Startup / Zéro Gaspillage (Flux Tendu)

5.2 Stratégie tarifaire (Produits d'Appel, Lissage de marge)

5.3 Rentabilité par panier et projection mensuelle

5.4 Ecosystème de paiement et stratégie d'acquisition

5.5 Gestion des risques opérationnels

Chapitre 6 : Gestion de Projet

6.1 Méthodologie adoptée (Agile)

6.2 Gestion des tâches avec GitHub

6.2.1 Utilisation du tableau Kanban

6.2.2 Gestion des issues et pull requests

6.2.3 Roadmap

6.3 Répartition des tâches dans l'équipe

6.4 Planning et Diagramme de Gantt

Chapitre 7 : Résultats et Evaluation

7.1 Bilan du projet

7.2 Difficultés rencontrées et solutions

7.3 Compétences acquises

Conclusion et Perspectives

Bibliographie

Annexes

A. Architecture technique détaillée

B. Captures d'écran de l'application

D. Guide d'installation et déploiement

E. Business Plan détaillé

Liste Des Abréviations

Abréviation Signification complète

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
B2B	Business to Business
B2C	Business to Consumer
CMI	Centre Monétique Interbancaire (passerelle de paiement marocaine)
COD	Cash On Delivery (paiement à la livraison)
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DAO	Data Access Object
DNS	Domain Name System
DOM	Document Object Model
DTO	Data Transfer Object
E2E	End-to-End
ESISA	École Supérieure d'Ingénierie en Sciences Appliquées

FK	Foreign Key (clé étrangère)
GPS	Global Positioning System
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IA	Intelligence Artificielle
IP	Internet Protocol
JIT	Just-In-Time (agrégation des besoins au plus juste)
JS	JavaScript
JSON	JavaScript Object Notation
JSONB	JavaScript Object Notation Binary (type JSON binaire de PostgreSQL)
JWT	JSON Web Token
KPI	Key Performance Indicator
LLM	Large Language Model
MCD	Modèle Conceptuel de Données
MLD	Modèle Logique de Données
ML	Machine Learning
MVC	Model-View-Controller
NPM	Node Package Manager
OAuth	Open Authorization
ORM	Object-Relational Mapping
OTP	One-Time Password (code de vérification à usage unique)
PFA	Projet de Fin d'Année
PK	Primary Key (clé primaire)
RAM	Random Access Memory

RBAC Role-Based Access Control (contrôle d'accès basé sur les rôles)

REST	Representational State Transfer
RGPD	Règlement Général sur la Protection des Données
ROI	Return On Investment
SaaS	Software as a Service
SDK	Software Development Kit
SIRET	Système d'Identification du Répertoire des Établissements
SMS	Short Message Service
SQL	Structured Query Language
SSL	Secure Sockets Layer
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TVA	Taxe sur la Valeur Ajoutée
UI	User Interface
UML	Unified Modeling Language
URI	Uniform Resource Identifier
URL	Uniform Resource Locator
UUID	Universally Unique Identifier
UX	User Experience
VPS	Virtual Private Server
XML	eXtensible Markup Language

Liste Des figures

Figure 1 : Logo et identité visuelle de SOUKI	20
Figure 2 : Fiche signalétique du projet SOUKI	20
Figure 3 : Chaîne d'approvisionnement traditionnelle vs modèle SOUKI	21
Figure 4 : Architecture fonctionnelle globale du système	27
Figure 5 : Architecture technique backend — FastAPI	39
Figure 6 : Architecture technique frontend — Next.js / React	40
Figure 7 : Architecture en couches du backend	41
Figure 8 : Extrait du cas d'utilisation Client — authentification	29
Figure 9 : Extrait du cas d'utilisation Client — commande	30
Figure 10 : Extrait du cas d'utilisation Administrateur	31
Figure 11 : Séquence d'inscription et de validation OTP	32
Figure 12 : Séquence du panier intelligent avec mécanisme de repli	33
Figure 13 : Séquence d'agrégation JIT régionale	34
Figure 14 : Page d'accueil de l'application SOUKI	35
Figure 15 : Catalogue et assistant de composition du panier	36
Figure 16 : Écran de sélection du profil de connexion	44
Figure 17 : Écran de connexion au back-office	37
Figure 18 : Cas d'utilisation complet — Client	62
Figure 19 : Cas d'utilisation complet — Parent	63
Figure 20 : Cas d'utilisation complet — Fournisseur	64
Figure 21 : Cas d'utilisation complet — Livreur	65
Figure 22 : Cas d'utilisation complet — Administrateur	65
Figure 23 : Séquence complète — Checkout et commande COD	66
Figure 24 : Séquence complète — Dispatch quotidien	67
Figure 25 : Séquence complète — Livraison et encaissement COD	68

Figure 26 : Diagramme de classes métier global	69
Figure 27 : Classes — Identité, profils et RBAC	69
Figure 28 : Classes — Catalogue, paniers et commandes	70
Figure 29 : Classes — Fournisseurs et offres	71
Figure 30 : Classes — JIT, dispatch et livraison	71
Figure 31 : Classes — Paiement, Wallet, COD et SAV	72
Figure 32 : Modèle relationnel simplifié de SOUKI	28

Liste Des Tableaux

Tableau 1 : Exigences non-fonctionnelles	XX
Tableau 2 : Comparaison des technologies backend	XX
Tableau 3 : Comparaison des technologies frontend	XX
Tableau 4 : Comparaison des bases de données	XX
Tableau 5 : Acteurs du système et leurs rôles	XX
Tableau 6 : Matrice des rôles et permissions (RBAC)	XX
Tableau 7 : Dictionnaire de données (synthèse des tables)	XX
Tableau 8 : Statuts du cycle de vie d'une commande	XX
Tableau 9 : Planning de développement par phase	XX
Tableau 10 : Métriques de performance et de qualité	XX

Introduction générale

Introduction Générale

Le Maroc dispose d'un secteur agricole particulièrement riche, représentant une part significative de son économie nationale et constituant le principal moyen de subsistance de millions de familles rurales. Pourtant, malgré cette abondance productive, la chaîne d'approvisionnement des produits frais — fruits, légumes, herbes aromatiques — reste dominée par des circuits traditionnels longs,

parents, et sources de pertes importantes à chaque maillon. Le consommateur urbain marocain se retrouve ainsi contraint de choisir entre deux options imparfaites : se déplacer dans les souks traditionnels, chronophages et physiquement contraignants, ou recourir à des applications de livraison généralistes dont les marges élevées se répercutent directement sur le prix final.

C'est de cette problématique concrète, ancrée dans le quotidien des ménages marocains, qu'est né SOUKI — une plateforme web innovante qui repense de fond en comble la relation entre le consommateur urbain et les produits frais. En s'appuyant sur un modèle dit de "circuit court", SOUKI élimine les intermédiaires inutiles : les produits sont approvisionnés à l'aube directement auprès des grossistes, triés et conditionnés le matin même, puis livrés à domicile avant midi. Ce modèle "Zéro Gaspillage" (flux tendu / Just-In-Time) garantit à la fois la fraîcheur maximale des produits et une facture systématiquement inférieure aux prix du marché de détail.

Sur le plan technologique, SOUKI adopte une architecture full-stack articulée autour de FastAPI/Python pour l'API, Next.js/React pour l'interface web et React Native/Expo pour l'application mobile. Le dépôt contient une authentification JWT avec OTP et Google OAuth, une autorisation RBAC, une commande texte ou vocale assistée par Gemini, un générateur de panier utilisant un modèle Gemma-2 distant avec repli local, une agrégation JIT planifiée à 20 h 00, ainsi qu'un dispatch automatique planifié à 21 h 35.

Le projet comprend également une réflexion économique fondée sur le flux tendu, la segmentation des produits et le contrôle des marges. Les valeurs de rentabilité présentes dans les documents internes constituent toutefois des scénarios prévisionnels : elles ne doivent pas être assimilées à des résultats commerciaux tant qu'elles n'ont pas été confirmées par des données réelles d'exploitation.

Ce rapport de Projet de Fin d'Années présente l'ensemble du parcours de conception, de réalisation et d'évaluation de cette solution. Il s'articule autour de sept chapitres complémentaires :

- Le Chapitre 1 contextualise le secteur agroalimentaire marocain et définit la problématique à laquelle SOUKI répond.
- Le Chapitre 2 détaille l'analyse des besoins, l'état de l'art comparatif, et la conception globale du système (diagrammes UML, maquettes, architecture fonctionnelle).

Le Chapitre 3 justifie les choix technologiques retenus pour le backend, le frontend, le mobile et les services tiers.

- Le Chapitre 4 présente la réalisation concrète de chaque module fonctionnel, des captures d'écran à l'implémentation technique.
- Le Chapitre 5 expose le modèle économique Lean Startup, la stratégie tarifaire et les projections financières.
- Le Chapitre 6 décrit la gestion de projet adoptée, la méthodologie Agile, et l'organisation de l'équipe.
- Le Chapitre 7 dresse un bilan objectif des résultats obtenus, des difficultés rencontrées et des compétences acquises.

Enfin, une conclusion générale ouvre des perspectives d'évolution pour SOUKI, notamment l'expansion géographique, le renforcement du modèle IA, et la structuration juridique en SARL pour accueillir des investisseurs.

Chapitre I : Contexte et problématique

Introduction

Ce chapitre situe le projet SOUKI dans son contexte métier et précise le problème auquel répond la solution. L'analyse est volontairement séparée de l'évaluation économique : les éléments techniques sont vérifiés dans le dépôt, tandis que les données de marché et les hypothèses financières devront être consolidées par une étude terrain.

1.1 Présentation du secteur agroalimentaire au Maroc

Les fruits et légumes frais occupent une place centrale dans les habitudes de consommation marocaines. Leur distribution repose sur une succession d'acteurs — producteurs, marchés de gros, détaillants, transporteurs et points de vente — dont la coordination influence le prix, la fraîcheur et la disponibilité. Pour une plateforme numérique, la difficulté ne se limite donc pas à afficher un catalogue : elle consiste à synchroniser la demande, l'achat, la préparation et la livraison de produits périssables.

SOUKI aborde ce contexte comme un problème de système d'information logistique. Le code contient des modèles dédiés aux produits, paniers, commandes, fournisseurs, zones JIT, tournées, livreurs, paiements et événements de livraison. Cette modélisation confirme que le projet traite simultanément le parcours commercial et l'exécution opérationnelle.

[À FAIRE : INSÉRER ICI UNE SOURCE STATISTIQUE OFFICIELLE RÉCENTE SUR LE SECTEUR AGRICOLE MAROCAIN]

1.2 Problématique : le compromis frais / prix / temps

L'utilisateur urbain recherche trois bénéfices qui peuvent entrer en tension : recevoir des produits frais, payer un prix maîtrisé et réduire le temps consacré à l'achat. Un stock important facilite la disponibilité mais augmente le risque d'invendus ; un approvisionnement tardif réduit ce risque mais impose une coordination stricte des commandes ; une livraison rapide devient coûteuse si les tournées sont peu denses.

La problématique retenue est ainsi formulée : comment concevoir une plateforme capable de collecter les demandes, de les transformer en besoins d'approvisionnement, d'affecter les commandes à des fournisseurs et à des livreurs, puis de suivre leur exécution tout en conservant la traçabilité des décisions ?

1.3 Solution proposée : SOUKI



Figure 1 : Logo et identité visuelle de SOUKI

Fiche signalétique du projet SOUKI	
Domaine	AgriTech / commerce et logistique de produits frais
Canaux	Application web/PWA et application mobile Expo
Backend	FastAPI, Python, SQLAlchemy, PostgreSQL
Frontend	Next.js, React, TypeScript, Tailwind CSS
Acteurs	Client, Parent, Fournisseur, Livreur, Administrateur
Fonctions clés	Catalogue, panier, IA, checkout, JIT, dispatch, wallet
Équipe	BOURKKADI IDRISSE Taha, ZMAROU Hamza, SGHIRI Salah Eddine
Établissement	ESISA — Projet de Fin d'Année

Figure 2 : Fiche signalétique du projet SOUKI

SOUKI propose une chaîne numérique intégrée. Le client consulte un catalogue, constitue un panier manuellement ou à l'aide d'une saisie intelligente, choisit une adresse et valide son checkout. Les commandes

confirmées sont agrégées par le processus JIT ; elles peuvent être rattachées à une zone et à un fournisseur, verrouillées, puis intégrées dans des tournées. Le livreur enregistre les événements opérationnels et le back-office supervise commandes, clients, fournisseurs, tarification, exceptions et indicateurs.

La solution n'est pas uniquement une vitrine e-commerce. Son intérêt réside dans la continuité entre l'intention d'achat et les opérations logistiques, continuité matérialisée par les services backend, les états de commande et les relations de la base PostgreSQL.

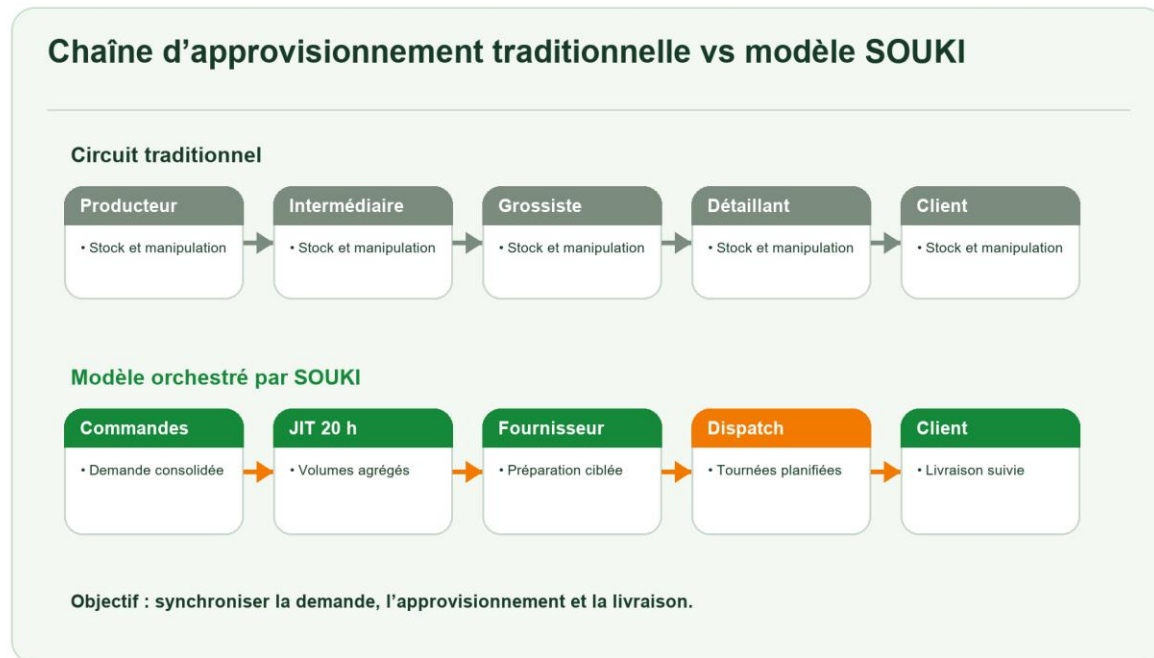


Figure 3 : Chaîne d'approvisionnement traditionnelle vs modèle SOUKI

1.4 Objectifs du projet

1.4.1 Objectifs techniques

- Construire une API modulaire séparant contrôleurs, services, DAO, DTO et entités.
- Fournir une interface web responsive et une application mobile partageant les mêmes contrats métier.
- Sécuriser l'accès par JWT, OTP, Google OAuth et contrôle d'accès fondé sur les rôles et permissions.
- Automatiser l'agrégation JIT, le verrouillage des commandes et la génération des tournées.
- Conserver une traçabilité des sessions, paiements COD, événements de livraison, réclamations et exécutions JIT.

1.4.2 Objectifs fonctionnels et business

- Simplifier la commande de produits frais depuis un catalogue numérique.

- Permettre la saisie manuelle, textuelle ou vocale d'un besoin.

- Donner aux fournisseurs, livreurs et administrateurs des espaces adaptés à leurs responsabilités.
- Réduire l'incertitude d'approvisionnement grâce à l'agrégation des demandes.
- Tester un modèle économique de flux tendu ; les marges et seuils restent des hypothèses à valider sur le terrain.

Conclusion

Le contexte justifie une solution qui coordonne commerce électronique et logistique de produits périssables. Le chapitre suivant transforme cette intention en besoins, risques, architectures et modèles de données vérifiables.

CHAPITRE 2 : ANALYSE ET CONCEPTION

Ce chapitre formalise les acteurs, les scénarios, les contraintes et la conception du système à partir du code et des documents du dépôt.

2.1 Analyse des besoins fonctionnels

2.1.1 Identification des acteurs (Client, Parent, Livreur, Fournisseur, Admin)

Acteur	Responsabilités observées dans le code
Client	Catalogue, panier, checkout, historique, profil, adresses, réclamations et wallet.
Parent	Accès à un espace dédié et modèle d'abonnement reliant parent et enfant.
Livreur	Consultation de tournée, ramassage, événements de livraison et validation COD.
Fournisseur	Demande d'adhésion, profil, offres produits, préparation et suivi des commandes.
Administrateur	Supervision, RBAC, produits, prix, clients, blacklist, fournisseurs, zones, JIT et dispatch.

2.1.2 Scenarios d'usage type

1. Un client s'inscrit, vérifie son OTP, se connecte, choisit ses produits, renseigne une adresse puis valide la commande.
2. Un client transmet une demande textuelle ou audio ; Gemini structure les éléments, le service les rapproche du catalogue et crée un brouillon.
3. À 20 h 00, le scheduler déclenche l'agrégation JIT ; les commandes éligibles sont regroupées, affectées à un fournisseur et verrouillées.
4. À 21 h 35, le dispatch construit les tournées du lendemain à partir des commandes verrouillées, des fournisseurs et des livreurs disponibles.
5. Le livreur exécute une tournée, déclare les événements et confirme le paiement à la livraison lorsque le mode COD est utilisé.

2.1.3 Cas d'utilisation détaillés

Le cas « valider un checkout » vérifie le client, son statut de blacklist pour le COD, le panier, l'adresse et le mode de paiement avant de créer la commande. Le cas « traiter une réclamation » associe l'utilisateur, la

commande et une ligne de panier, calcule le remboursement et crédite SoukiWallet. Le cas « administrer un fournisseur » utilise un cycle de validation et des permissions spécifiques. Ces cas montrent que la logique métier est placée dans les services, les contrôleurs se limitant à exposer les contrats HTTP.

2.1.4 Exigences non-fonctionnelles

Exigence	Réponse technique
Sécurité	Hachage bcrypt, JWT signé, OTP haché, Google OAuth et RBAC.
Maintenabilité	Architecture en couches, interfaces, DTO et services spécialisés.
Traçabilité	Logs JIT, sessions, transactions wallet, événements de livraison et confirmations COD.
Portabilité	API REST consommée par Next.js et React Native/Expo.
Résilience	Repli local pour le panier ML et jobs planifiés journalisés ; validation terrain encore requise.

2.2 Etat de l'art / Analyse comparative du marché

2.2.1 Applications de livraison généralistes (Glovo, Kaalix, Khodrati)

Ces solutions privilégient l'agrégation d'offres et la rapidité de livraison. La comparaison fonctionnelle détaillée, les tarifs et les zones couvertes doivent être confirmés par une étude de marché datée.

2.2.2 Plateformes E-commerce bio (Green Village)

Les plateformes spécialisées valorisent la qualité et la sélection de produits. SOUKI s'en distingue techniquement par la présence d'un workflow JIT et d'un dispatch intégré, sans qu'une supériorité commerciale puisse être affirmée sans données comparatives.

2.2.3 Marchés traditionnels physiques (Souks)

Le souk offre proximité culturelle, choix et négociation, mais impose un déplacement et une disponibilité temporelle. SOUKI cherche à numériser une partie de cette expérience sans prétendre remplacer toutes ses dimensions.

2.2.4 Notre différenciation (valeur ajoutée)

- Agrégation JIT planifiée et historisée.

- **Chaîne** multi-rôles reliant client, fournisseur, livreur et administration.

- Commande texte/audio adaptée au français et à la darija.
- Gestion des zones, des tournées, des anomalies et du paiement COD.
- Application web/PWA et client mobile Expo reposant sur la même API.

2.3 Analyse des risques du projet

Risque	Impact	Mesure observée ou recommandée
Dépendance aux services IA	Panier vocal indisponible	Fallback local présent pour Gemma ; message d'erreur contrôlé pour Gemini.
Qualité des adresses	Dispatch incomplet	Coordonnées et adresse par défaut ; validation opérationnelle nécessaire.
Paiement carte non intégré	Parcours limité	Option CMI affichée mais désactivée ; ne pas la présenter comme disponible.
Données périssables	Rupture ou gaspillage	Buffer JIT de 10 % ; valeur à calibrer avec des données réelles.
Sécurité du token navigateur	Exposition en cas de XSS	JWT et RBAC présents ; migration vers cookie HttpOnly recommandée.

2.4 Spécifications techniques

2.4.1 Vue d'ensemble de l'architecture

Le système comprend une API FastAPI, une base PostgreSQL via SQLAlchemy, un frontend Next.js, un client mobile Expo, un stockage Supabase pour les médias et plusieurs services externes optionnels.

2.4.2 Architecture fonctionnelle (modules métier)

Les modules confirmés sont : authentification et autorisation, profil et paramètres, catalogue et pricing, panier et commande intelligente, checkout, commandes, JIT, dispatch, livraison, fournisseurs, administration, réclamations, blacklist, notifications et wallet.

2.4.3 Architecture technique (Backend / Frontend / Mobile)

Le backend applique le flux Contrôleur → Service → DAO → Entité. Le frontend utilise l'App Router de Next.js, des composants réutilisables et des hooks. L'application mobile utilise Expo Router, React Query, Zustand, SecureStore et Mapbox.

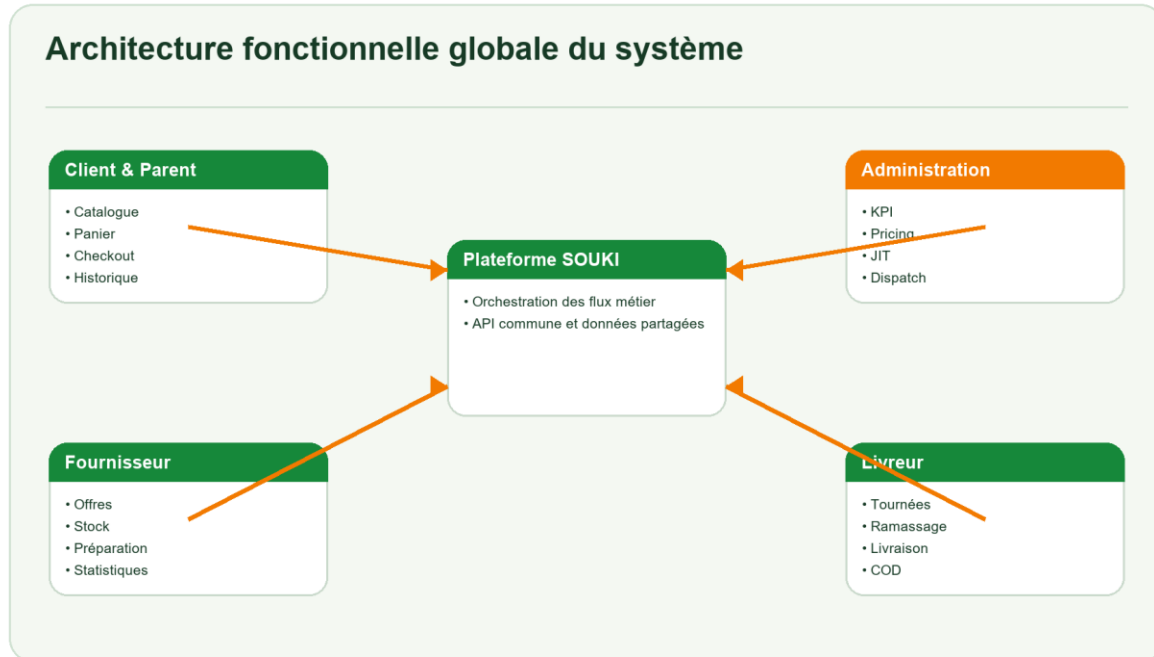


Figure 4 : Architecture fonctionnelle globale du système

2.5 Architecture globale du système

2.5.1 Workflows détaillés (agrégation JIT, commande vocale, dispatch)

Le JIT sélectionne les commandes EN_ATTENTE ou CONFIRMEE du jour, agrège les quantités par produit, ajoute un buffer de 10 %, arrondit au kilogramme supérieur, résout zone et fournisseur, puis verrouille les commandes. La commande vocale transmet l'audio encodé à Gemini, récupère un JSON, résout les produits et crée le brouillon. Le dispatch répartit les commandes verrouillées entre les livreurs et ordonne les arrêts par proximité à partir des coordonnées disponibles.

2.5.2 Gestion des données

SQLAlchemy fournit le modèle principal d'exécution. Prisma décrit un sous-ensemble du schéma pour l'outillage. Des scripts SQL versionnés complètent les synchronisations automatiques de schéma au démarrage.

2.5.3 Intégrations externes

Les intégrations vérifiées sont Google OAuth, Google Gemini, Hugging Face Inference, Supabase Storage, SMTP et Mapbox. L'intégration CMI/PayZone n'est pas opérationnelle dans le checkout actuel.

2.5.4 Sécurité et authentification

Les mots de passe et codes OTP sont hachés par bcrypt. Les accès API utilisent des JWT. Les permissions RBAC protègent les espaces métier, tandis que Google OAuth valide les jetons d'identité auprès de Google.

2.6 Conception de la base de données (modèles PostgreSQL)

Le schéma relationnel comprend notamment t_users, t_clients, t_parents, t_livres, t_fournisseurs, T_Product, t_paniers, t_lignes_panier, t_commandes, t_paiements, t_tournees, t_delivery_events, t_zones_jit, t_jit_logs, t_claims, wallets, t_transactions_wallet, roles, permissions, user_roles et role_permissions. Les clés étrangères relient profils, commandes, paniers, produits, tournées et acteurs.

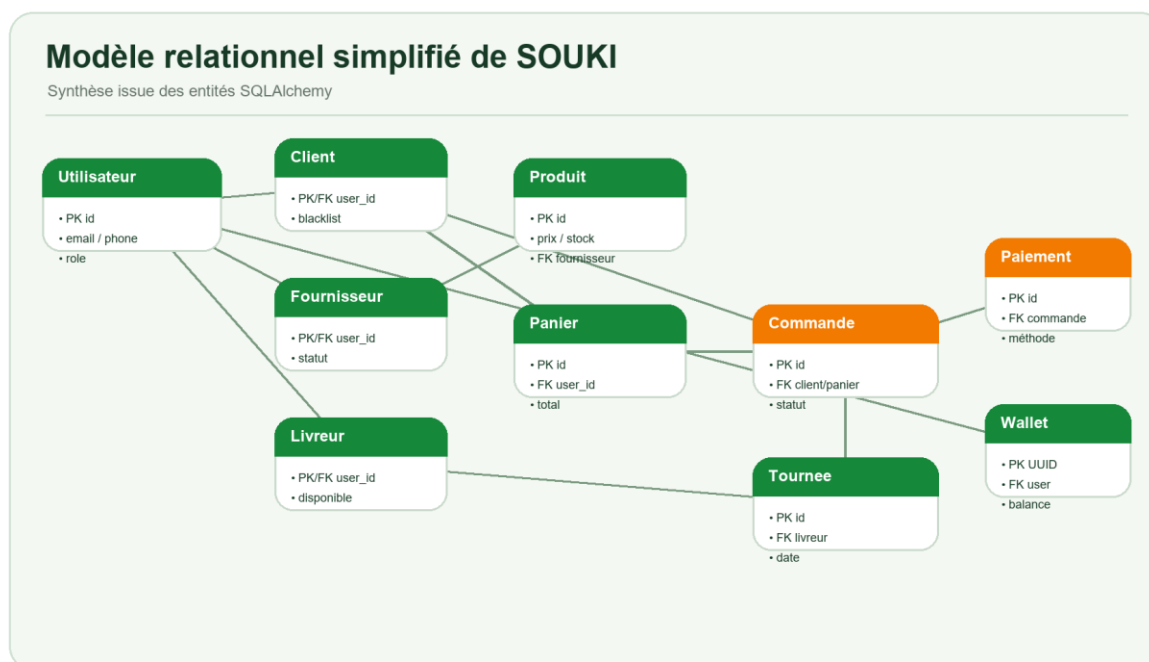


Figure 32 : Modèle relationnel simplifié de SOUKI

2.7 Diagrammes UML (Cas d'utilisation, Séquences)

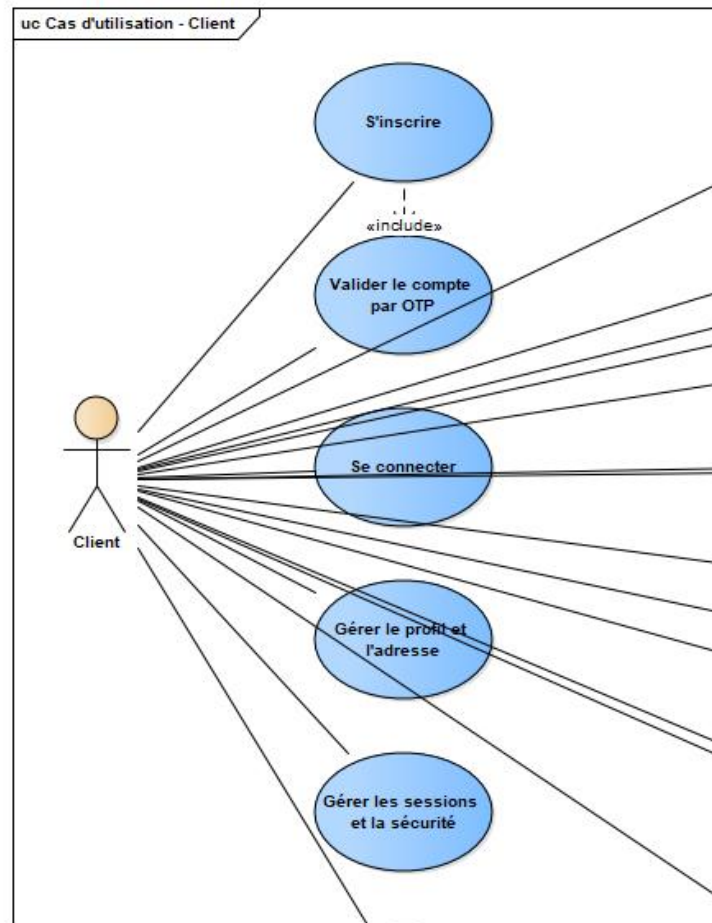


Figure 8 : Extrait du cas d'utilisation Client — authentication

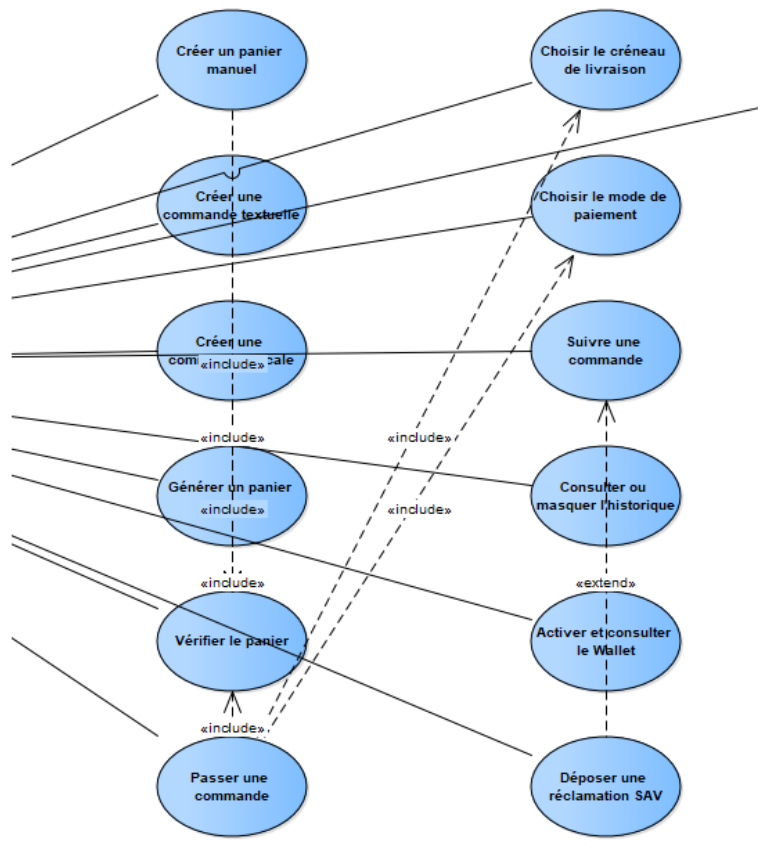


Figure 9 : Extrait du cas d'utilisation Client — commande



Figure 10 : Extrait du cas d'utilisation Administrateur

2.7.1 Authentification classique (Email/Téléphone + OTP)

L'inscription crée un utilisateur non vérifié, hache le mot de passe, génère un OTP à six chiffres, hache ce code et l'envoie par email lorsqu'un SMTP est configuré. La vérification active le compte et émet un JWT.

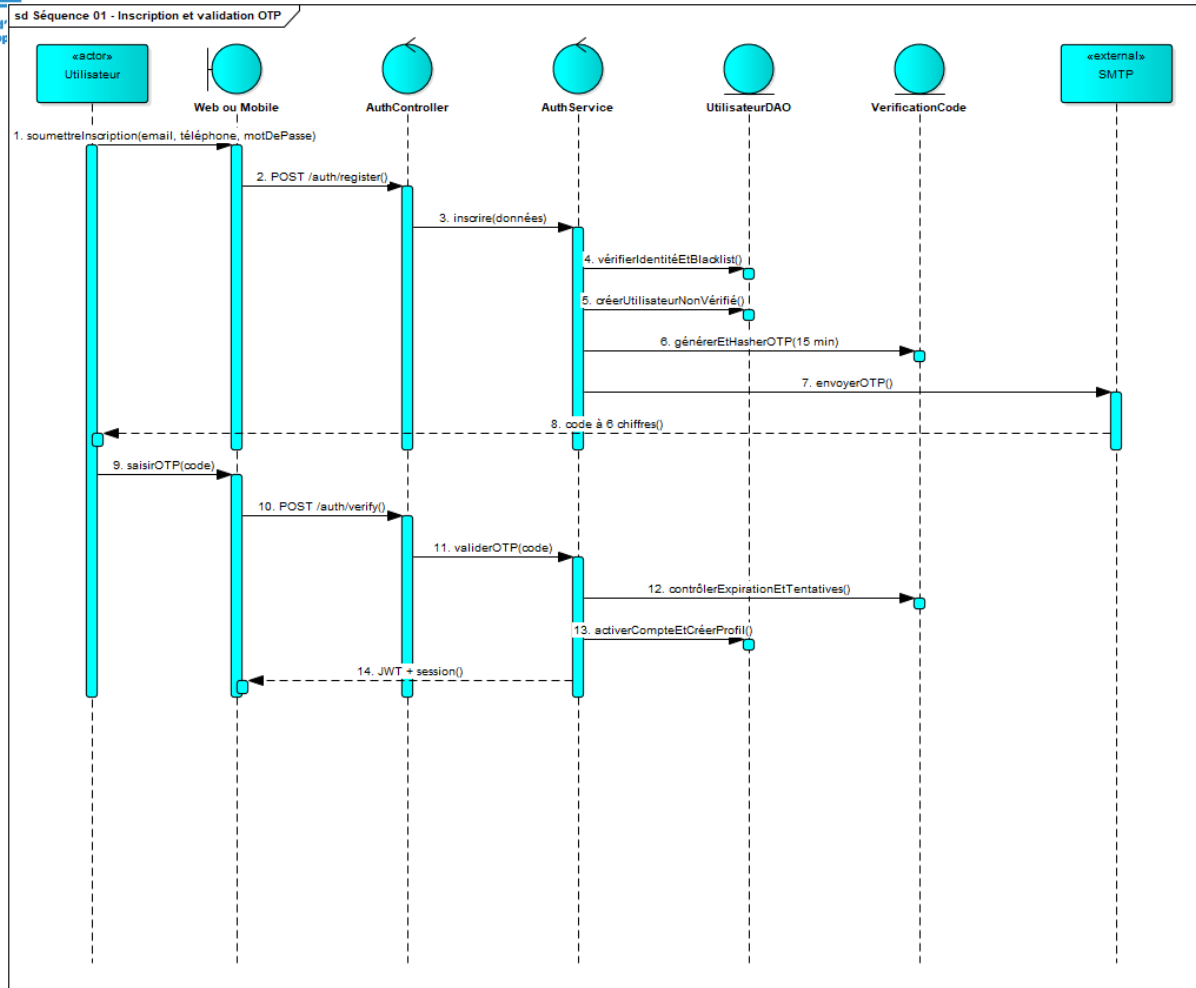


Figure 11 : Séquence d'inscription et de validation OTP

2.7.2 S'authentifier via OAuth (Google)

Le frontend obtient un jeton Google, le backend le valide avec le client Google configuré, crée ou met à jour l'utilisateur, applique ses rôles, puis émet son propre JWT.

Le flux Google OAuth est décrit textuellement ; son diagramme détaillé pourra être ajouté après validation de la configuration de production.

2.7.3 Créer une commande vocale

Le fichier audio est contrôlé, encodé en Base64, envoyé à Gemini avec un prompt structuré, puis les lignes reconnues sont rapprochées du catalogue et persistées.

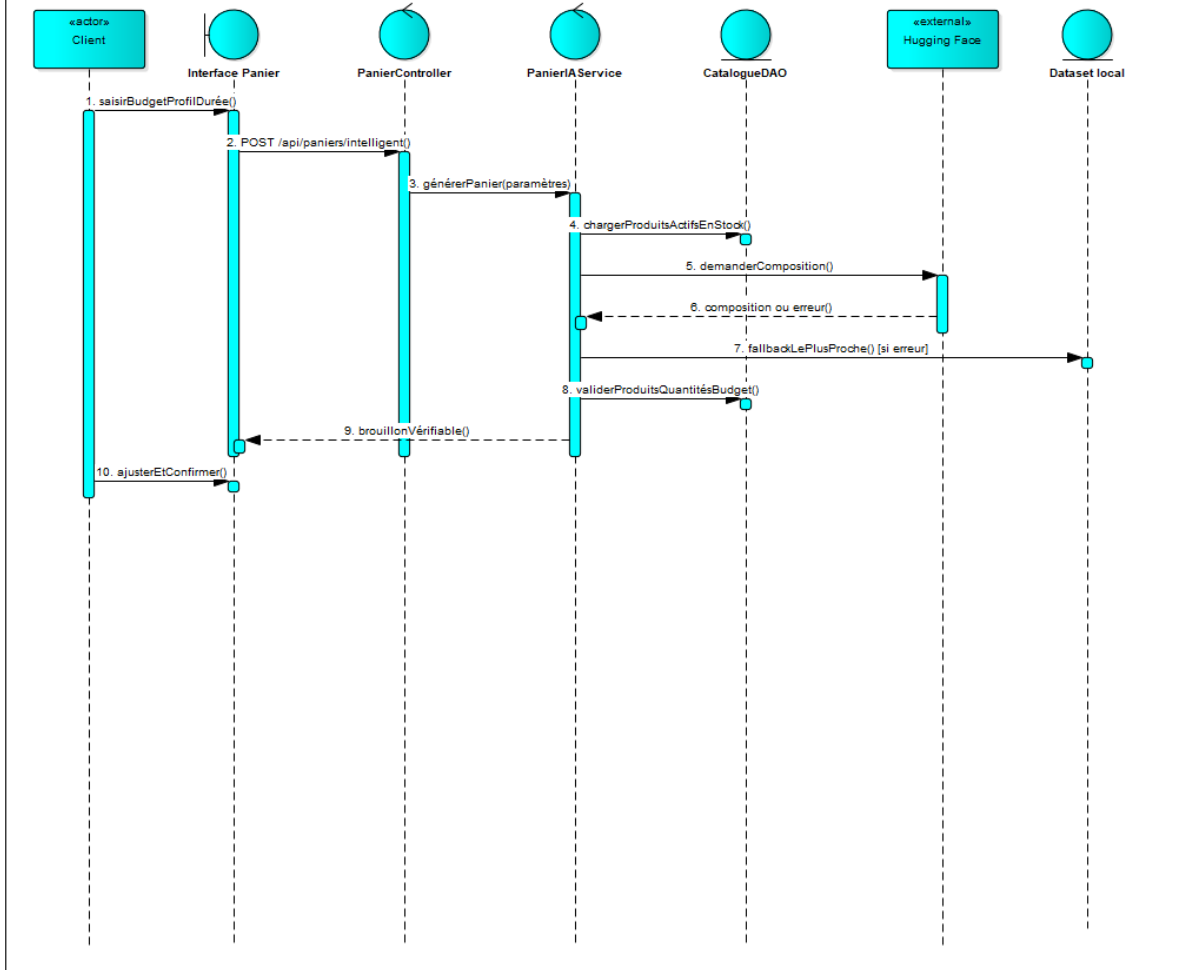


Figure 12 : Séquence du panier intelligent avec mécanisme de repli

2.7.4 Processus d'agrégation JIT

Le scheduler déclenche le service, les quantités sont agrégées, un fournisseur est résolu, les commandes sont verrouillées et un log d'exécution est enregistré.

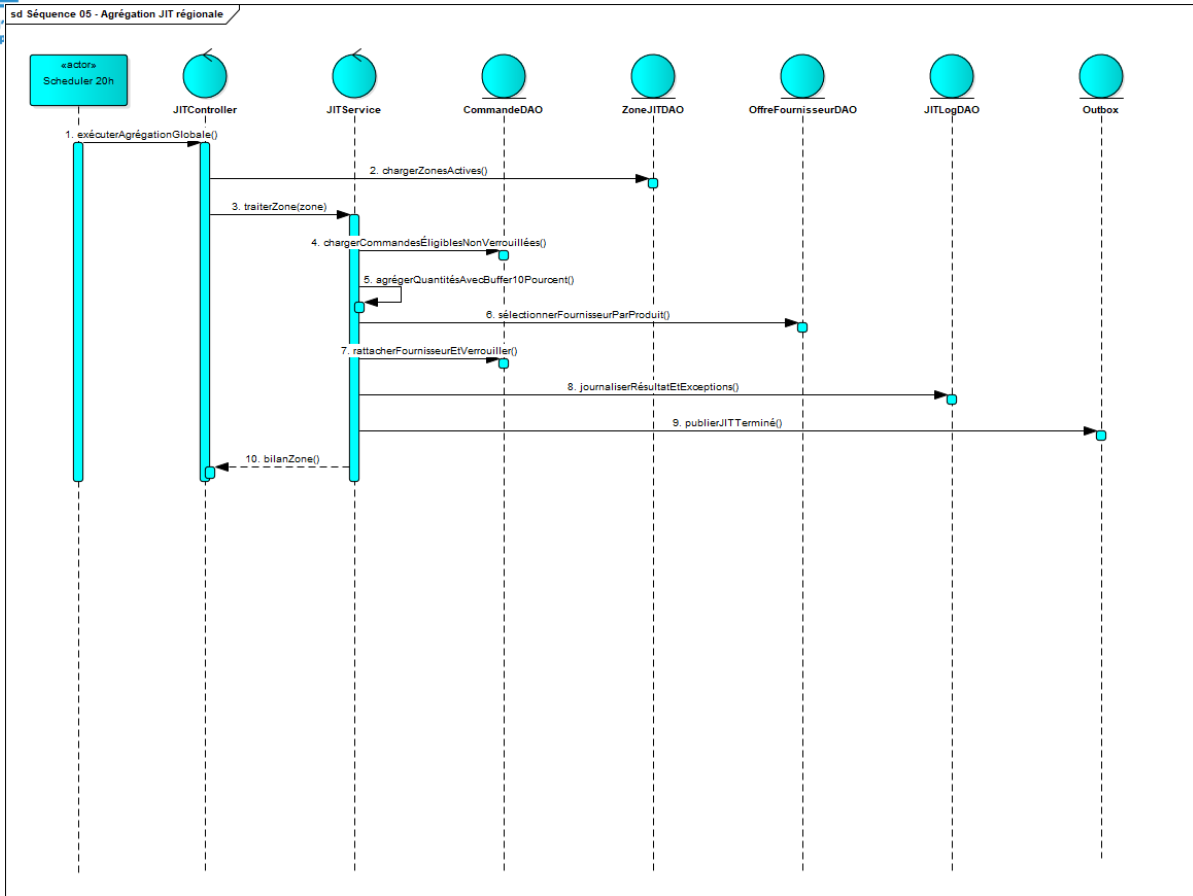


Figure 13 : Séquence d'agrégation JIT régionale

2.8 Maquettes et prototypes de l'interface

2.8.1 Parcours Client

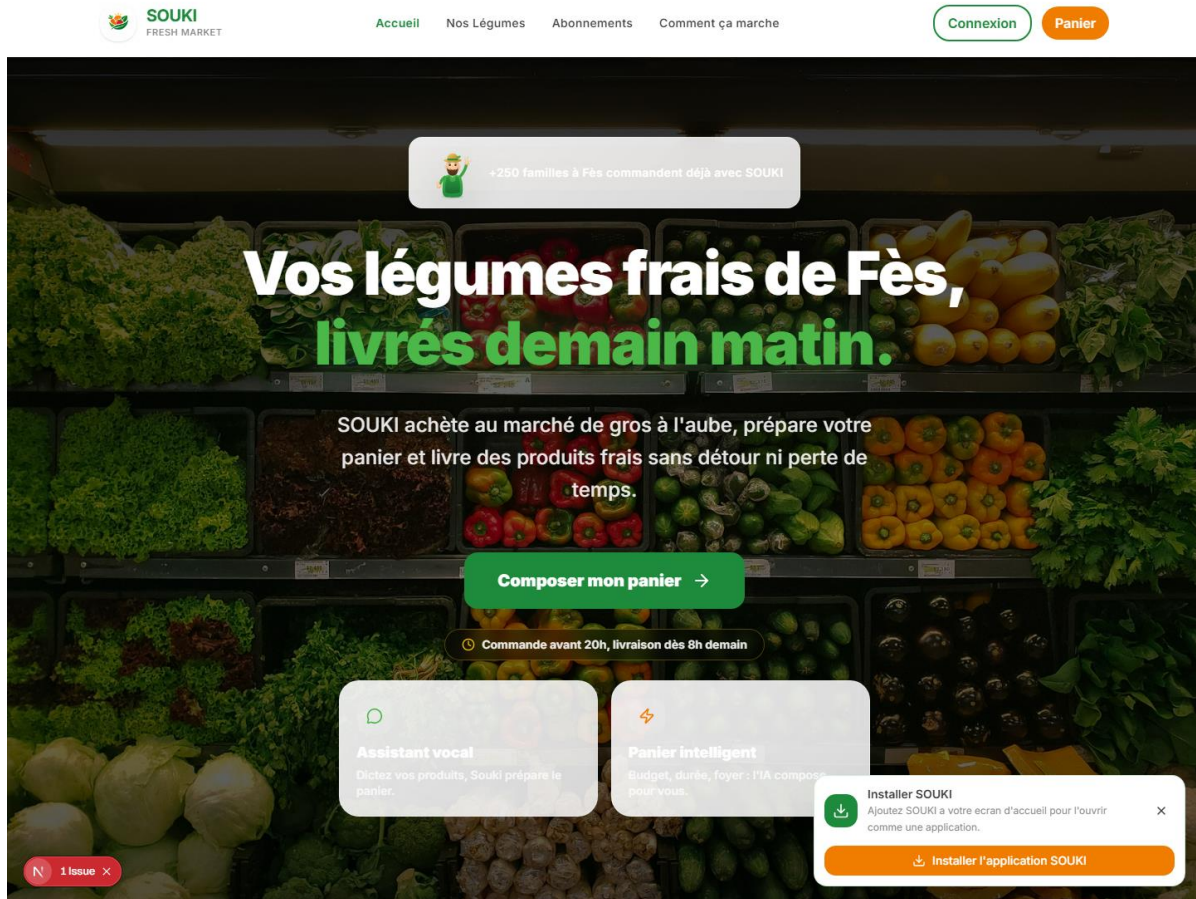


Figure 14 : Page d'accueil de l'application SOUKI

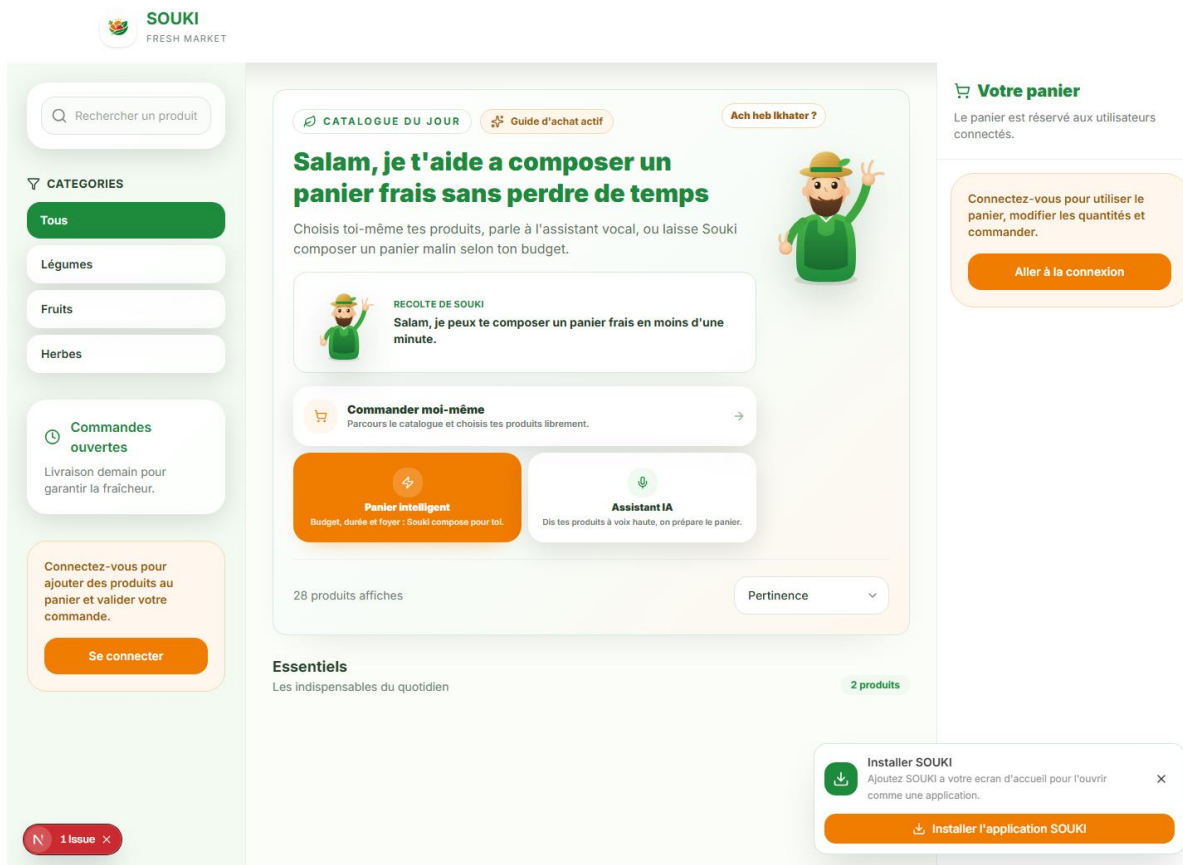


Figure 15 : Catalogue et assistant de composition du panier

2.8.2 Parcours Livreur

Les écrans opérationnels du livreur nécessitent une session authentifiée et des tournées affectées ; ils sont à capturer lors de la démonstration.

2.8.3 Parcours Fournisseur

Les écrans fournisseur nécessitent une session approuvée et des offres actives ; ils sont à capturer lors de la démonstration.

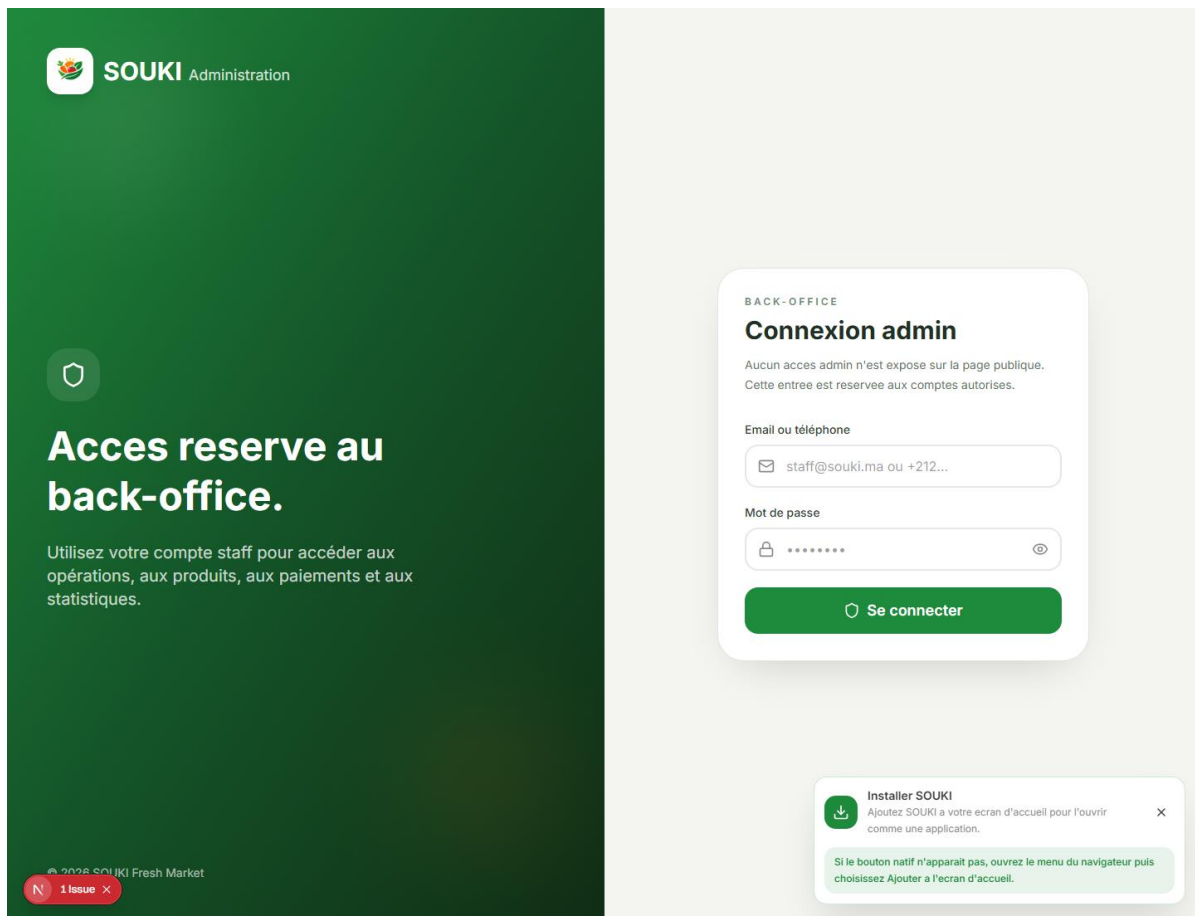


Figure 17 : Écran de connexion au back-office

Conclusion

La conception montre une couverture fonctionnelle large et une architecture cohérente. Les diagrammes et captures doivent être produits depuis la version réellement exécutée de l'application.

CHAPITRE 3 : CHOIX TECHNOLOGIQUES ET ARCHITECTURE TECHNIQUE

Introduction

Les choix technologiques sont évalués selon leur rôle réel dans le dépôt, sans confondre dépendance déclarée et service effectivement disponible.

3.1 Architecture Full-Stack

3.1.1 Architecture Backend (FastAPI/Python)

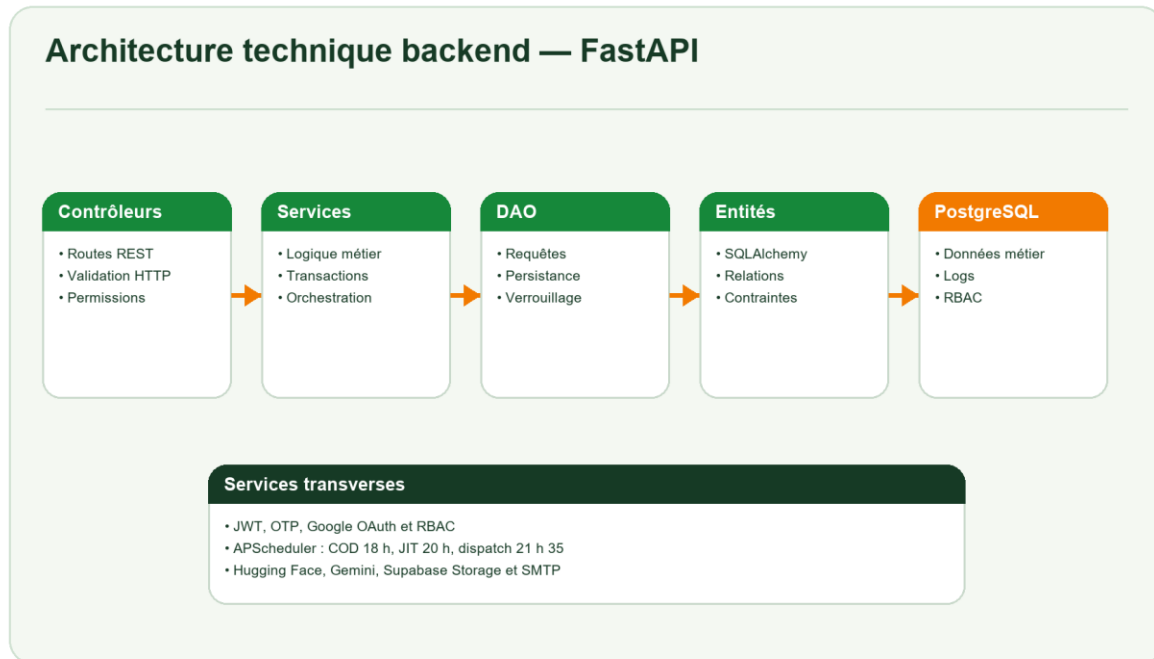


Figure 5 : Architecture technique backend — FastAPI

FastAPI expose les routeurs REST, valide les entrées et injecte les dépendances. SQLAlchemy assure la persistance PostgreSQL. APScheduler orchestre l’alerte COD à 18 h 00, le JIT à 20 h 00 et le dispatch à 21 h 35, dans le fuseau Africa/Casablanca.

3.1.2 Architecture Frontend (Next.js/React)

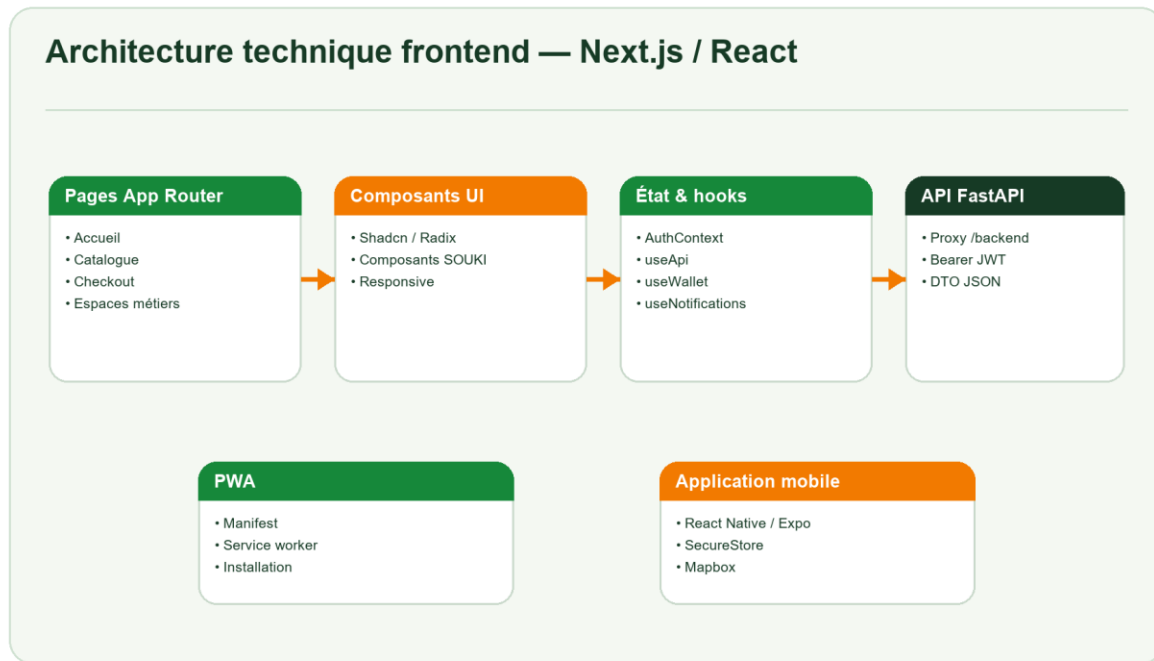


Figure 6 : Architecture technique frontend — Next.js / React

Le frontend utilise Next.js 16, React 19 et TypeScript. L'App Router organise les espaces client, livreur, fournisseur et administrateur. Les composants Radix/Shadcn, Tailwind CSS, Recharts et Mapbox fournissent l'interface et les visualisations.

3.1.3 Architecture Mobile (PWA)

Le dépôt comprend à la fois une PWA web — manifeste, service worker et invite d'installation — et une application mobile React Native/Expo. Il convient donc de parler d'un canal web installable et d'un client mobile natif multiplateforme.

S2 Justification des choix technologiques

3.2.1 Stack Backend (FastAPI, PostgreSQL, SQLAlchemy)

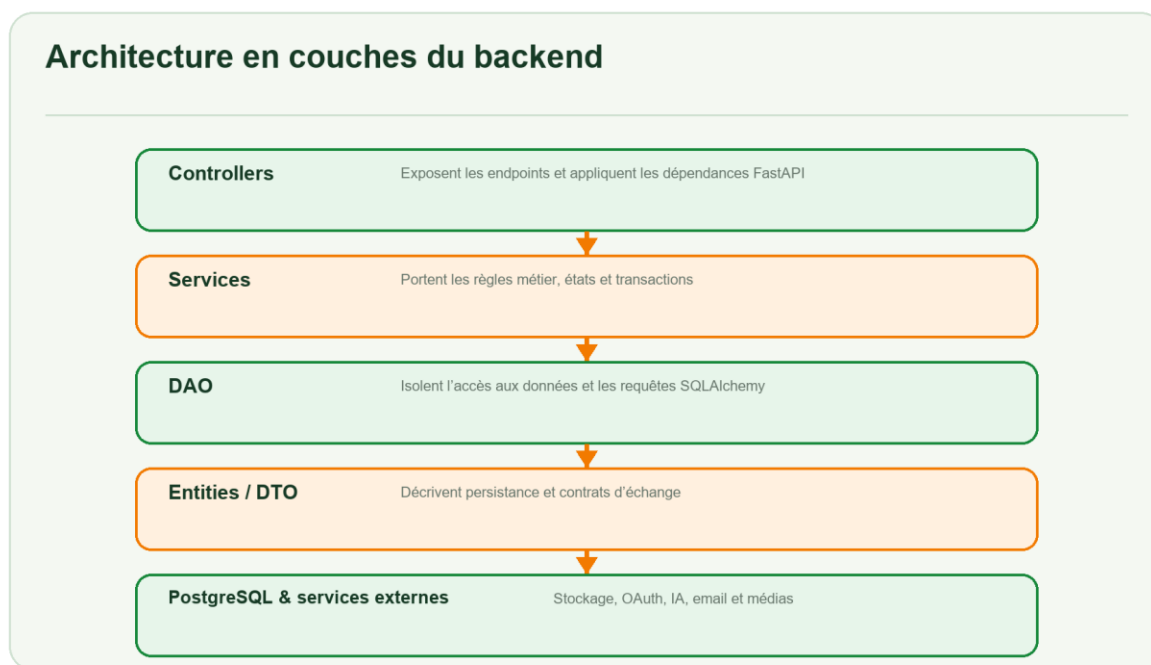


Figure 7 : Architecture en couches du backend

FastAPI convient à une API typée et modulaire ; PostgreSQL garantit un modèle relationnel adapté aux transactions ; SQLAlchemy permet d'exprimer relations, contraintes et requêtes métier en Python. Les interfaces DAO facilitent les tests et limitent le couplage.

3.2.2 Stack Frontend (Next.js, TypeScript, Shadcn/UI, TailwindCSS)

Next.js fournit routage, rendu et proxy backend. TypeScript réduit les incohérences de contrats. Les composants accessibles de Radix/Shadcn et les classes Tailwind accélèrent la construction d'écrans homogènes.

3.2.3 Services tiers (Hugging Face, Gemini AI, Google OAuth, Supabase, PayZone)

Service	Usage vérifié	Statut
Hugging Face	Inférence distante du modèle TahaBDI/gemma-2-2b-panier-merged.	Implémenté avec fallback local.
Gemini AI	Extraction structurée d'une commande texte ou audio.	Implémenté, dépend d'une clé API.

Google OAuth	Validation d'un ID token et création/connexion utilisateur.	Implémenté.
Supabase	PostgreSQL selon configuration et stockage des avatars/médias.	Implémenté selon variables d'environnement.
PayZone / CMI	Paielement par carte.	Non vérifié ; option CMI désactivée dans le checkout.

3.3 Sécurité et authentification

Le backend applique bcrypt pour les secrets, JWT pour l'authentification stateless et un modèle RBAC persistant. Les OTP expirent après quinze minutes, sont limités en tentatives et en renvois, et sont stockés sous forme hachée. Les routes utilisent require_auth ou require_permission. La configuration CORS est limitée aux origines déclarées. Pour une mise en production, la gestion du JWT côté navigateur devra être durcie et complétée par une politique de rotation, des cookies HttpOnly et des tests de sécurité.

Conclusion

La stack retenue est cohérente avec une application multi-clients et fortement orientée API. Les services externes restent découplés grâce aux variables d'environnement et aux mécanismes de repli.

CHAPITRE 4 : RÉALISATION ET IMPLÉMENTATION

Introduction

La réalisation est présentée module par module. Les descriptions ci-dessous correspondent aux contrôleurs, services, DAO, entités et écrans présents dans le dépôt.

4.1 Modules développés

4.1.1 Module d'authentification et gestion multi-rôles

Inscription email ou téléphone, OTP, connexion locale, Google OAuth, JWT, profils spécialisés et RBAC. Les rôles CLIENT, PARENT, LIVREUR, FOURNISSEUR et ADMIN déterminent les tableaux de bord et permissions.

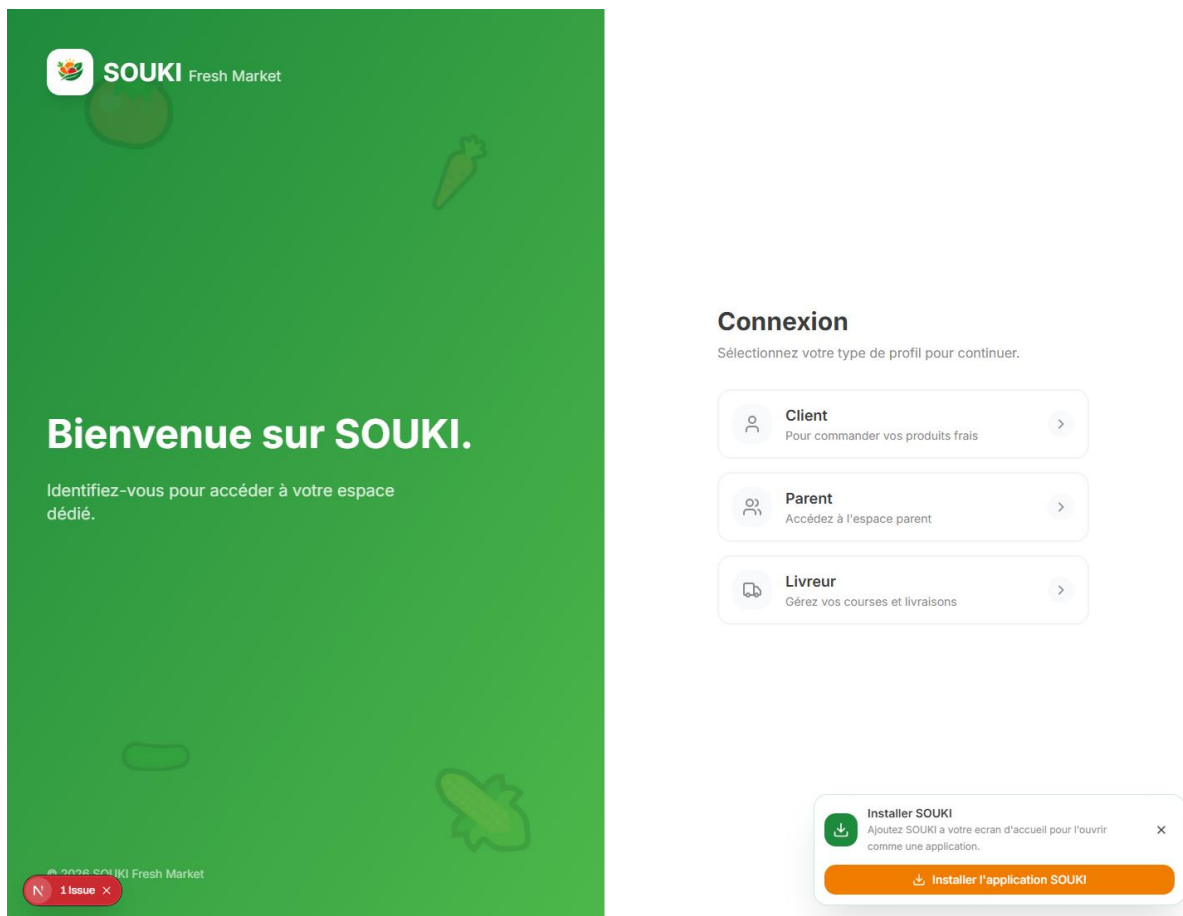
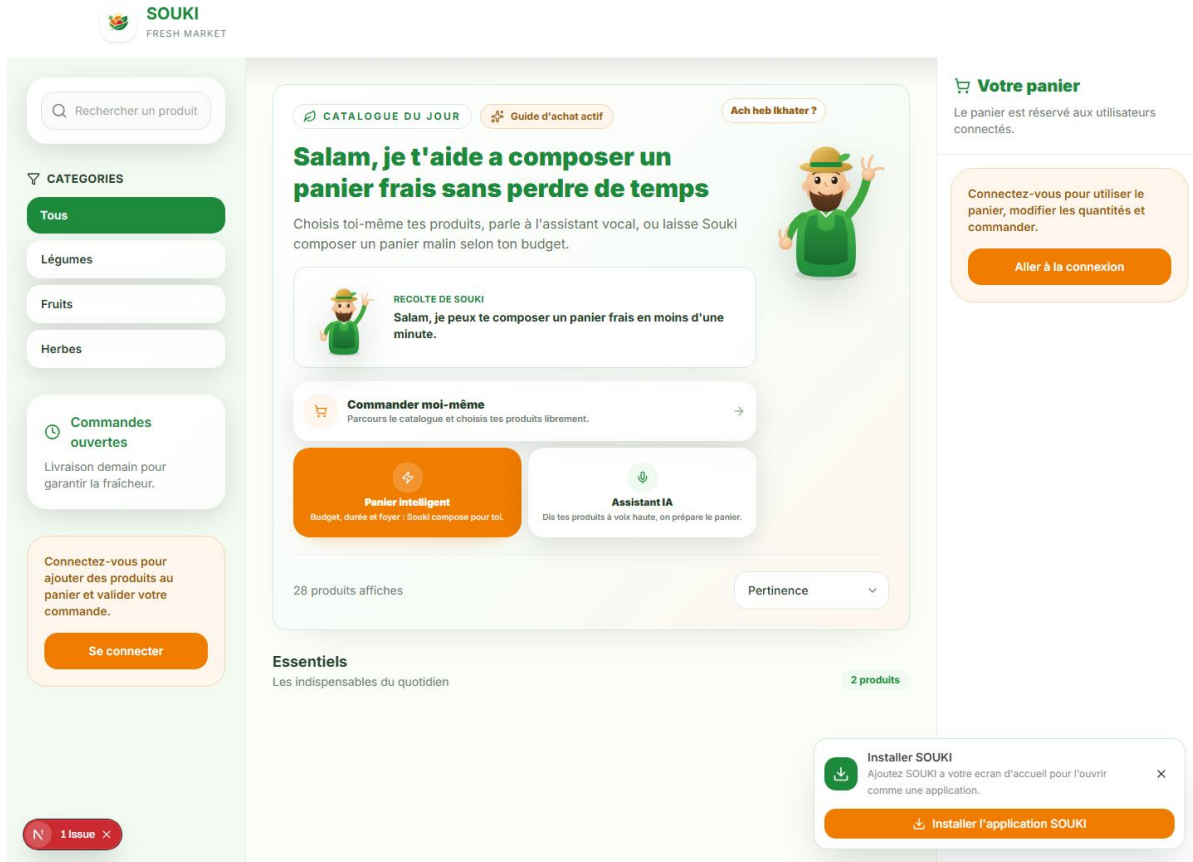


Figure 16 : Écran de sélection du profil de connexion

4.1.2 Module Catalogue et gestion des produits

Catalogue actif, noms français/darija, prix, stock, unité, image, niveau tarifaire et rattachement fournisseur. L'administration gère produits et paramètres de pricing.



Vue réelle du catalogue de produits

4.1.3 Module Panier (manuel, texte, vocal)

Ajout et modification des lignes, calcul des sous-totaux et frais de livraison, génération d'un panier par critères, ainsi que création de brouillons à partir de texte ou audio.

Le catalogue présenté à la figure 15 expose les entrées du panier manuel, du panier intelligent et de l'assistant vocal.

4.1.4 Module IA - Assistant vocal (Gemma-2-2B fine-tune QLoRA)

Deux chaînes coexistent : Gemini extrait les produits d'un texte ou d'un audio ; Gemma-2 génère une composition de panier depuis un budget, un nombre de personnes, une durée et un profil. Le dépôt prouve l'appel au modèle fusionné, mais pas le protocole d'entraînement QLoRA ; celui-ci doit être documenté par le notebook et les artefacts d'apprentissage.

L'interface publique de la figure 15 présente l'accès au panier intelligent et à l'assistant IA.

4.1.5 Module Checkout et paiement

Le checkout contrôle panier, adresse, créneau, blacklist COD et mode de paiement. Le COD et le wallet sont représentés. Le paiement CMI est annoncé dans l'interface mais désactivé et ne doit pas être présenté comme opérationnel.

4.1.6 Module Agrégation JIT

Agrégation quotidienne, buffer de 10 %, journalisation, zones régionales, résolution du fournisseur, verrouillage et déverrouillage administratif.

4.1.7 Module Dispatch et optimisation des tournées

Génération des tournées du lendemain, allocation des livreurs par fournisseur, équilibrage des lots et ordre de passage fondé sur une heuristique de proximité utilisant la distance de Haversine.

4.1.8 Module Livreur (tracking, événements de livraison)

Consultation de tournée, ramassage fournisseur, démarrage, événements de livraison, retours dépôt, anomalies et validation du paiement COD.

4.1.9 Module Fournisseur (marketplace B2B)

Demande fournisseur, validation administrative, profil boutique, offres produit, préparation des commandes et statistiques quotidiennes. Le terme marketplace décrit ici la relation multi-fournisseurs, non une place de marché publique complète.

4.1.10 Module Réclamations et Wallet

Une réclamation référence commande et ligne de panier. Le remboursement validé crédite le SoukiWallet et crée une transaction. La table wallets constitue la source moderne ; t_wallets reste un héritage de migration.

4.2 Dashboards spécialisés

Le client dispose du catalogue, du checkout, de l'historique et des paramètres. Le fournisseur suit ses produits, commandes et préparations. Le livreur visualise ses tournées et actions opérationnelles. L'administrateur consulte KPI, clients, commandes, produits, prix, fournisseurs, zones, livreurs, blacklist et exceptions.

4.3 Intégrations externes

Les intégrations sont isolées dans des services ou bibliothèques dédiés : Google pour OAuth, Gemini pour l'analyse multimodale, Hugging Face pour l'inférence ML, Supabase pour le stockage, SMTP pour les emails et Mapbox pour la cartographie. Chacune dépend de variables d'environnement et doit être testée dans le contexte de déploiement.

Conclusion

La réalisation couvre la chaîne principale de commande et de livraison. Les éléments encore incomplets sont explicitement identifiés afin d'éviter de confondre prototype fonctionnel et service de production.

CHAPITRE 5 : ETUDE FINANCIÈRE ET MODÈLE ECONOMIQUE

Introduction

Les documents internes décrivent une stratégie de prix et plusieurs simulations. Faute de factures d'achat, coûts salariaux consolidés, historique de ventes et relevés de paiement, ce chapitre les traite comme des hypothèses prévisionnelles.

5.1 Modèle Lean Startup / Zéro Gaspillage (Flux Tendu)

Le principe est de retarder l'approvisionnement jusqu'à la consolidation des commandes. Le JIT implémenté fournit le mécanisme technique : agrégation, buffer, verrouillage et liste d'achat. L'expression « zéro gaspillage » représente une cible ; le buffer de 10 % montre qu'une marge physique demeure prévue.

5.2 Stratégie tarifaire (Produits d'Appel, Lissage de marge)

Le modèle de pricing classe les produits en trois niveaux : produits d'appel, produits courants et produits à plus forte valeur. Les champs de marge cible, coussin de sécurité, prix de gros et prix affiché permettent une gestion différenciée. Les pourcentages proposés dans le document de stratégie doivent être calibrés à partir de prix réels et validés juridiquement et commercialement.

5.3 Rentabilité par panier et projection mensuelle

Une rentabilité exploitable devra calculer, par commande, le chiffre d'affaires encaissé, le coût d'achat réel, l'emballage, la livraison, les frais de paiement, les remboursements et les pertes. Les projections actuelles ne constituent pas des résultats. [INFORMATION NON VÉRIFIABLE DANS LE CODE SOURCE – VALIDATION HUMAINE REQUISE]

Indicateur à mesurer	Source attendue
Panier moyen	Commandes livrées et payées.
Marge brute produit	Prix de vente moins coût d'achat facturé.
Coût de livraison	Temps, distance, carburant et rémunération.
Taux de perte	Quantités achetées, livrées et jetées.
Coût d'acquisition	Dépenses marketing et nouveaux clients actifs.

5.4 Ecosystème de paiement et stratégie d'acquisition

Le code gère le COD, la confirmation téléphonique, la blacklist associée et SoukiWallet. Le CMI reste désactivé. Le parrainage est représenté dans le profil client, mais son coût d'acquisition et son efficacité n'ont pas été mesurés.

5.3 Gestion des risques opérationnels

- Variabilité du prix de gros : mise à jour fréquente et garde-fous de marge.
- Échec de livraison COD : confirmation, blacklist, événements et anomalies.
- Dépendance fournisseur : zones, offres et résolution par commande.
- Remboursements : crédit wallet traçable et détection de réclamations suspectes.
- Sous-dimensionnement logistique : mesure des distances et équilibrage des tournées.

Conclusion

Le modèle économique est techniquement instrumenté mais pas encore démontré. La prochaine étape consiste à réaliser un pilote et à remplacer chaque hypothèse par une mesure horodatée et vérifiable.

CHAPITRE 6 : GESTION DE PROJET

L'historique Git, les branches fonctionnelles, les pull requests fusionnées, les tests et la documentation témoignent d'un développement itératif réparti par domaines.

6.1 Méthodologie adoptée (Agile)

Le dépôt reflète une démarche incrémentale : authentification, catalogue, commande vocale, JIT, logistique, wallet, pricing, fournisseur et mobile ont été développés sur des branches distinctes puis intégrés. Cette organisation se rapproche d'une approche Agile par fonctionnalités.

6.2 Gestion des tâches avec GitHub

6.2.1 Utilisation du tableau Kanban

Les références aux missions, tâches restantes et checklists montrent un suivi par lots. Une capture du tableau GitHub Projects est néanmoins nécessaire pour prouver la structure exacte du Kanban.

[À FAIRE : INSÉRER ICI UNE CAPTURE DU TABLEAU KANBAN GITHUB PROJECTS]

6.2.2 Gestion des issues et pull requests

L'historique contient de nombreuses fusions de pull requests et des branches nommées par fonctionnalité. La convention favorise la traçabilité du domaine traité, même si certains messages de commit gagneraient à être plus descriptifs.

6.2.3 Roadmap

La roadmap observable progresse du socle d'authentification et de catalogue vers le checkout, le JIT, la logistique, l'administration, les fournisseurs et l'application mobile. Les prochaines étapes recommandées sont l'intégration réelle du paiement carte, la sécurisation production, la mesure de performance et le pilote métier.

6.3 Répartition des tâches dans l'équipe

Les fichiers fournis ne contiennent pas une matrice officielle reliant chaque membre à ses livrables.
[INFORMATION NON VÉRIFIABLE DANS LE CODE SOURCE – VALIDATION HUMAINE REQUISE]

**[À FAIRE : COMPLÉTER LA RÉPARTITION DES TÂCHES ENTRE BOURKKADI IDRISSE
TAHA, ZMAROU HAMZA ET SGHIRI SALAH EDDINE]**

6.4 Planning et Diagramme de Gantt

Les dates des migrations et des commits permettent de reconstruire une chronologie, mais elles ne remplacent pas un planning validé indiquant début, fin, dépendances et responsables.

Conclusion

La gestion par branches et intégrations successives a permis d'élargir progressivement le périmètre. La formalisation du Kanban, de la répartition et du Gantt reste à compléter avec les données de l'équipe.

CHAPITRE 7 : RÉSULTATS ET EVALUATION

L'évaluation distingue les résultats démontrables dans le dépôt des résultats nécessitant une exécution instrumentée ou un pilote utilisateur.

7.1 Bilan du projet

- API FastAPI structurée en couches et connectée à PostgreSQL.
- Interfaces web et mobile couvrant plusieurs rôles.
- Authentification locale, OTP, Google OAuth et RBAC.
- Chaîne catalogue, panier, checkout, commande, JIT, fournisseur, dispatch et livraison.
- Dix fichiers de tests backend ciblant notamment JIT, logistique, fournisseurs, zones et cloisonnement.
- Documentation technique importante, mais parfois plus ambitieuse que l'implémentation actuelle.

Aucune mesure fiable de charge, de disponibilité, de taux de conversion ou de satisfaction n'est archivée dans le dépôt. Ces indicateurs ne sont donc pas chiffrés.

7.2 Difficultés rencontrées et solutions

Difficulté	Réponse apportée
Coordination de nombreux rôles	RBAC, profils spécialisés et route guards.
Évolution du schéma	Scripts SQL et services de synchronisation au démarrage.
Disponibilité du modèle ML	Inférence Hugging Face avec fallback de compositions local.
Cohérence du cycle de commande	Machine à états et services de verrouillage/dispatch.
Historique wallet	Migration vers wallets UUID tout en conservant la compatibilité legacy.

7.3 Compétences acquises

- Conception d'API REST et architecture en couches.
- Modélisation relationnelle PostgreSQL avec SQLAlchemy.
- Développement Next.js/React et React Native/Expo.
- Authentification, OAuth, JWT, OTP et autorisation RBAC.
- Intégration de services IA, stockage, email et cartographie.

Conclusion

SOUKI constitue un prototype avancé et cohérent. Son passage à un produit exploitable dépend maintenant de la validation métier, de tests de sécurité et de charge, de l'instrumentation des KPI et d'un pilote opérationnel.

CONCLUSION ET PERSPECTIVES

Le projet SOUKI démontre la faisabilité technique d'une plateforme marocaine de commande et de distribution de produits frais. Son apport principal tient à l'intégration d'un parcours client, d'une agrégation JIT, d'un réseau fournisseur, d'un dispatch logistique et d'une supervision multi-rôles dans une architecture commune.

Les perspectives prioritaires sont l'activation d'un paiement carte réellement certifié, le renforcement de la sécurité des sessions, la consolidation des migrations, la génération des diagrammes depuis le schéma réel, la mise en place d'observabilité, les tests de charge et un pilote mesurant fraîcheur, pertes, ponctualité, marge et satisfaction. L'expansion géographique ne devra être envisagée qu'après validation de ces indicateurs.

BIBLIOGRAPHIE

1. Documentation officielle FastAPI — <https://fastapi.tiangolo.com/>
2. Documentation SQLAlchemy — <https://docs.sqlalchemy.org/>
3. Documentation Next.js — <https://nextjs.org/docs>
4. Documentation React — <https://react.dev/>
5. Documentation Expo — <https://docs.expo.dev/>
6. Documentation PostgreSQL — <https://www.postgresql.org/docs/>
7. Documentation Google Identity — <https://developers.google.com/identity>
8. Documentation Hugging Face Inference — <https://huggingface.co/docs>
9. Documentation Supabase — <https://supabase.com/docs>
10. Documentation Mapbox — <https://docs.mapbox.com/>
11. Dépôt SOUKI : code source, migrations SQL et documentation interne, version analysée le 23 juin 2026.

ANNEXES

A. Architecture technique détaillée

Voir les dossiers back-end, front-end, mobile, prisma et Documentation.

Conception UML complète

Cas d'utilisation par acteur

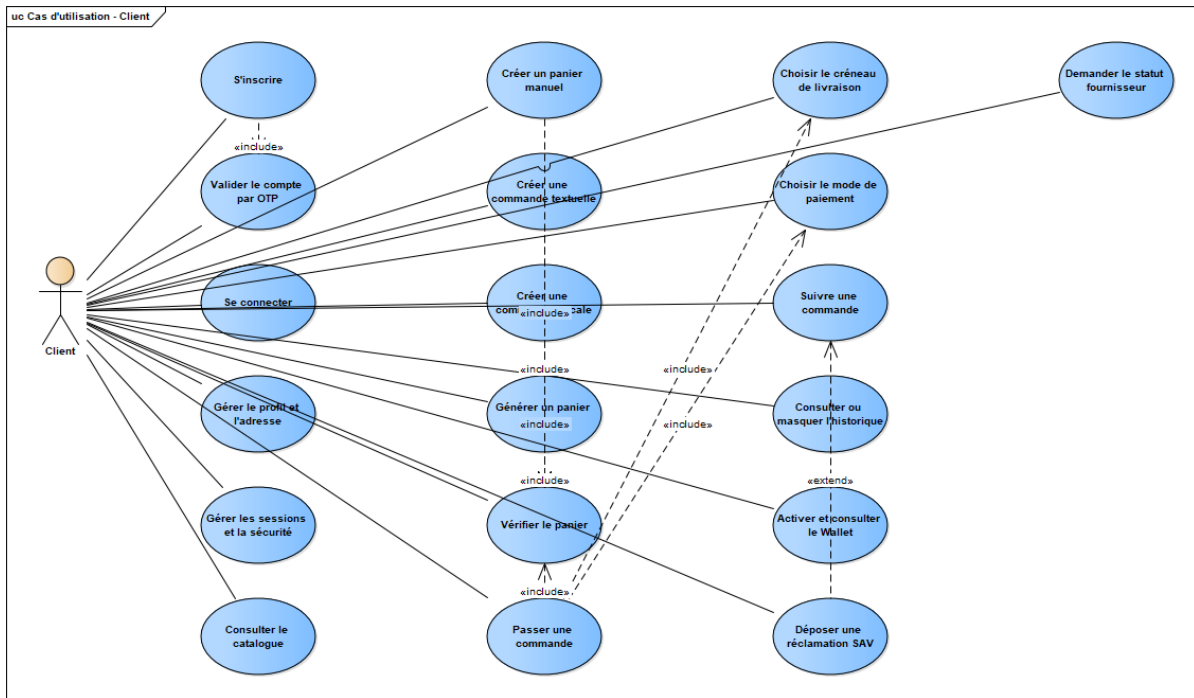


Figure 18 : Cas d'utilisation complet — Client

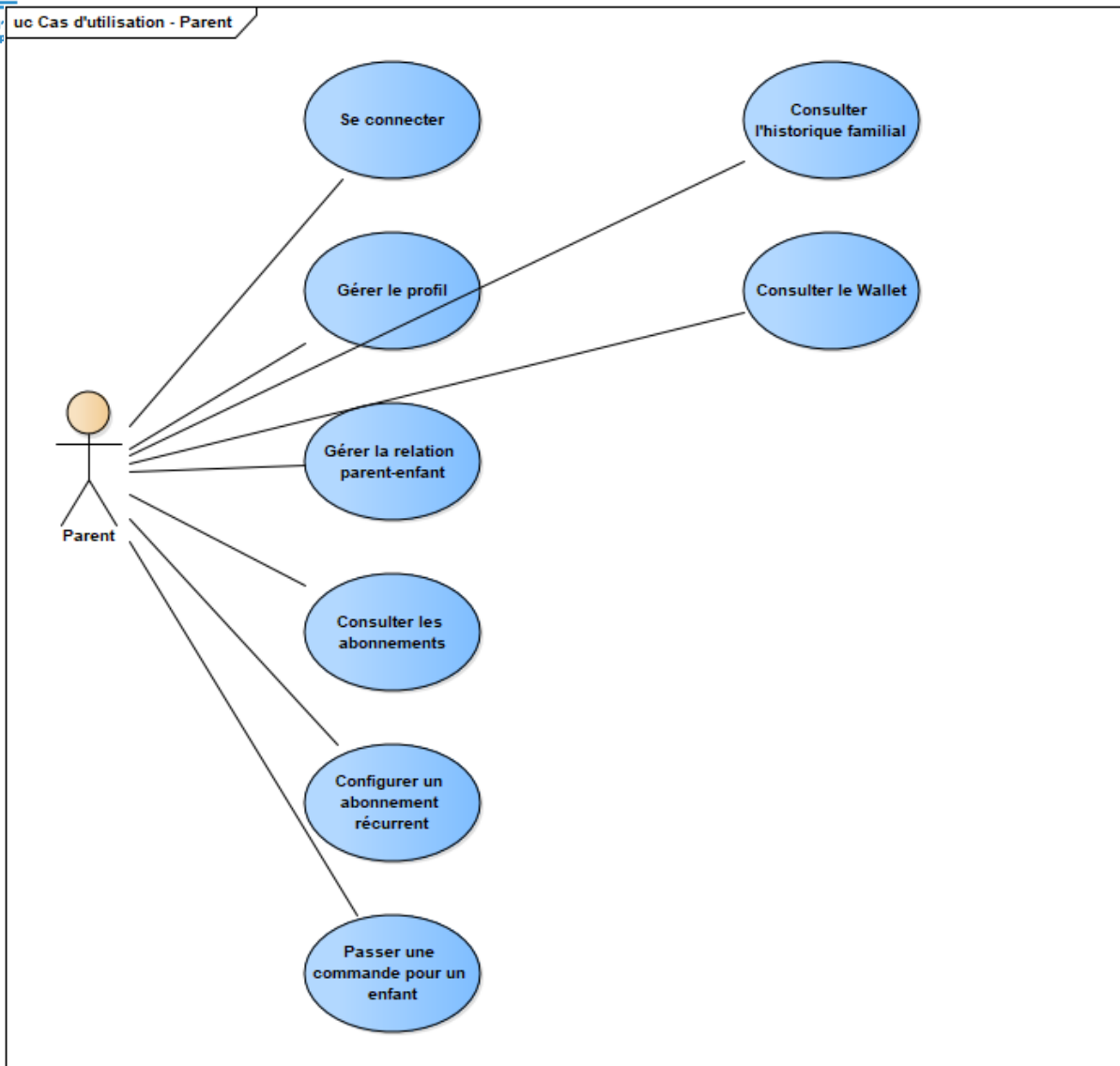


Figure 19 : Cas d'utilisation complet — Parent

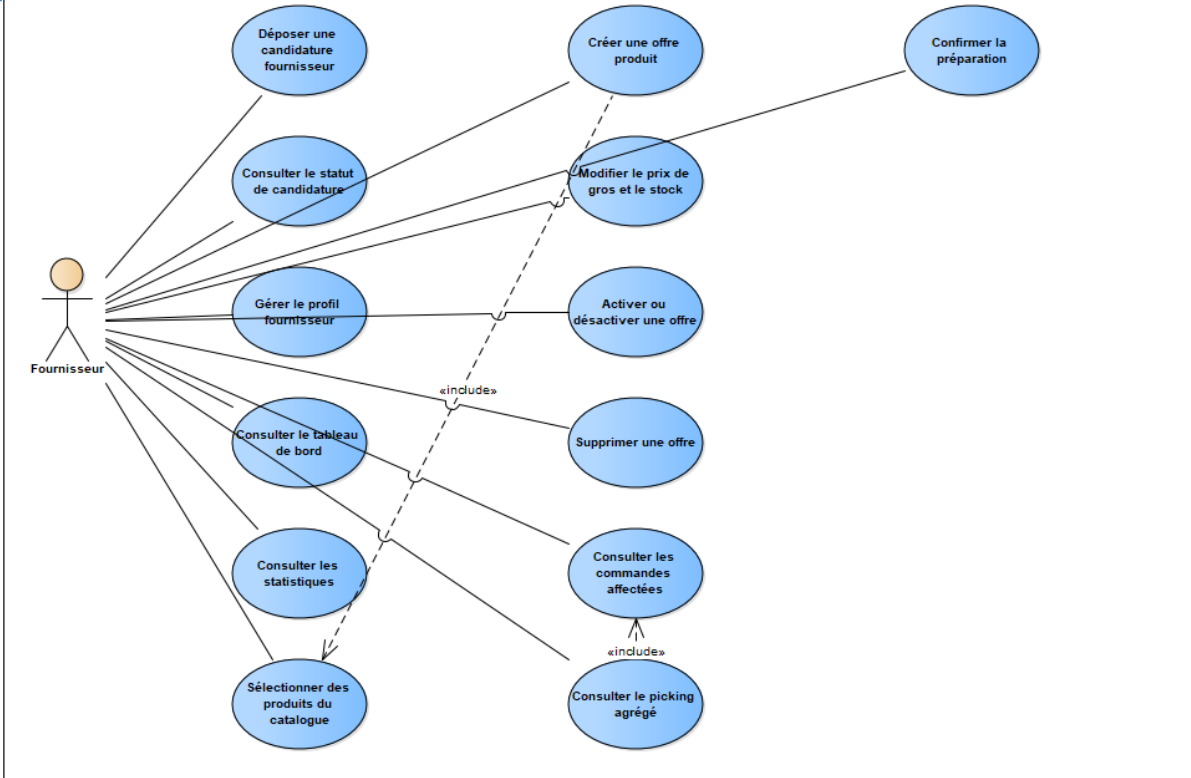


Figure 20 : Cas d'utilisation complet — Fournisseur

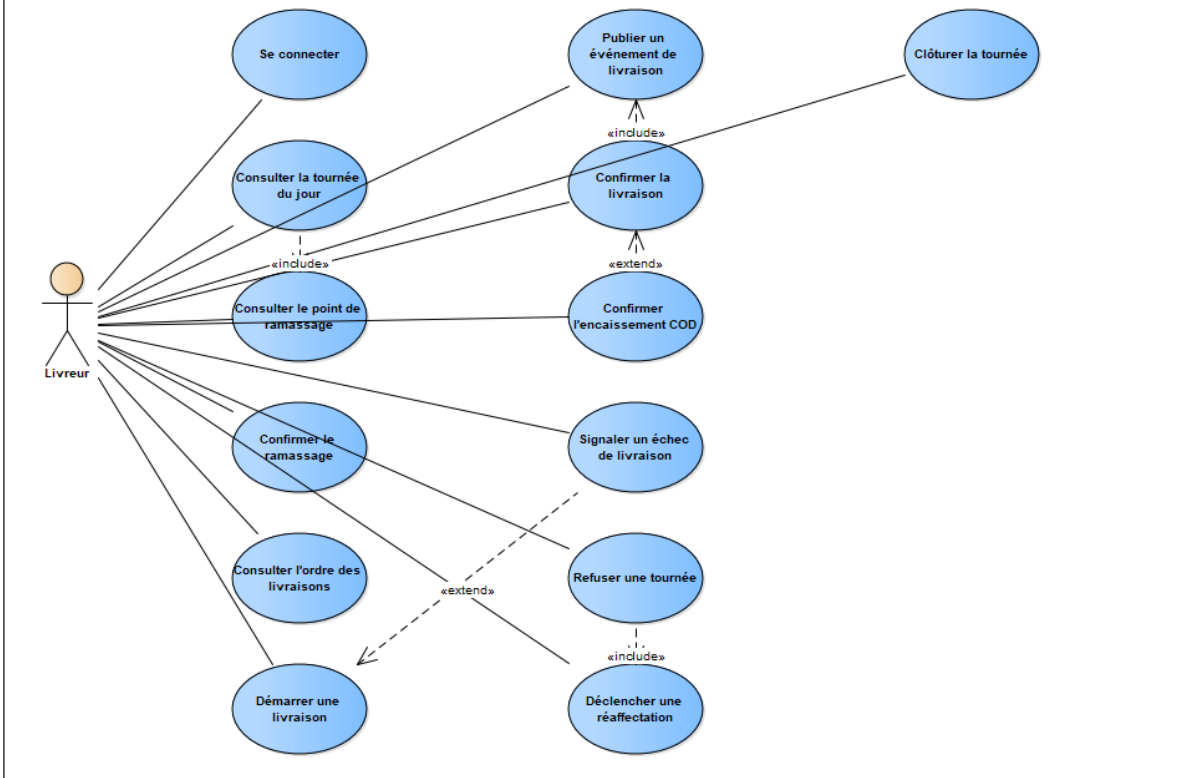


Figure 21 : Cas d'utilisation complet — Livreur

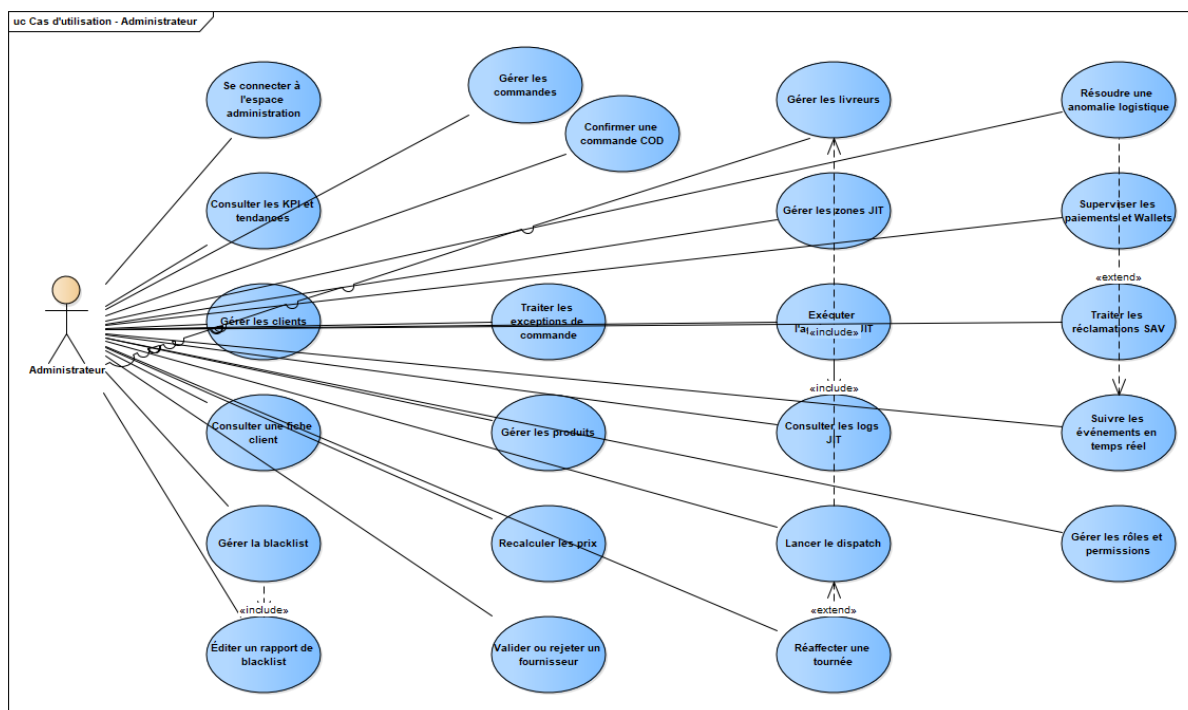


Figure 22 : Cas d'utilisation complet — Administrateur

Séquences complémentaires

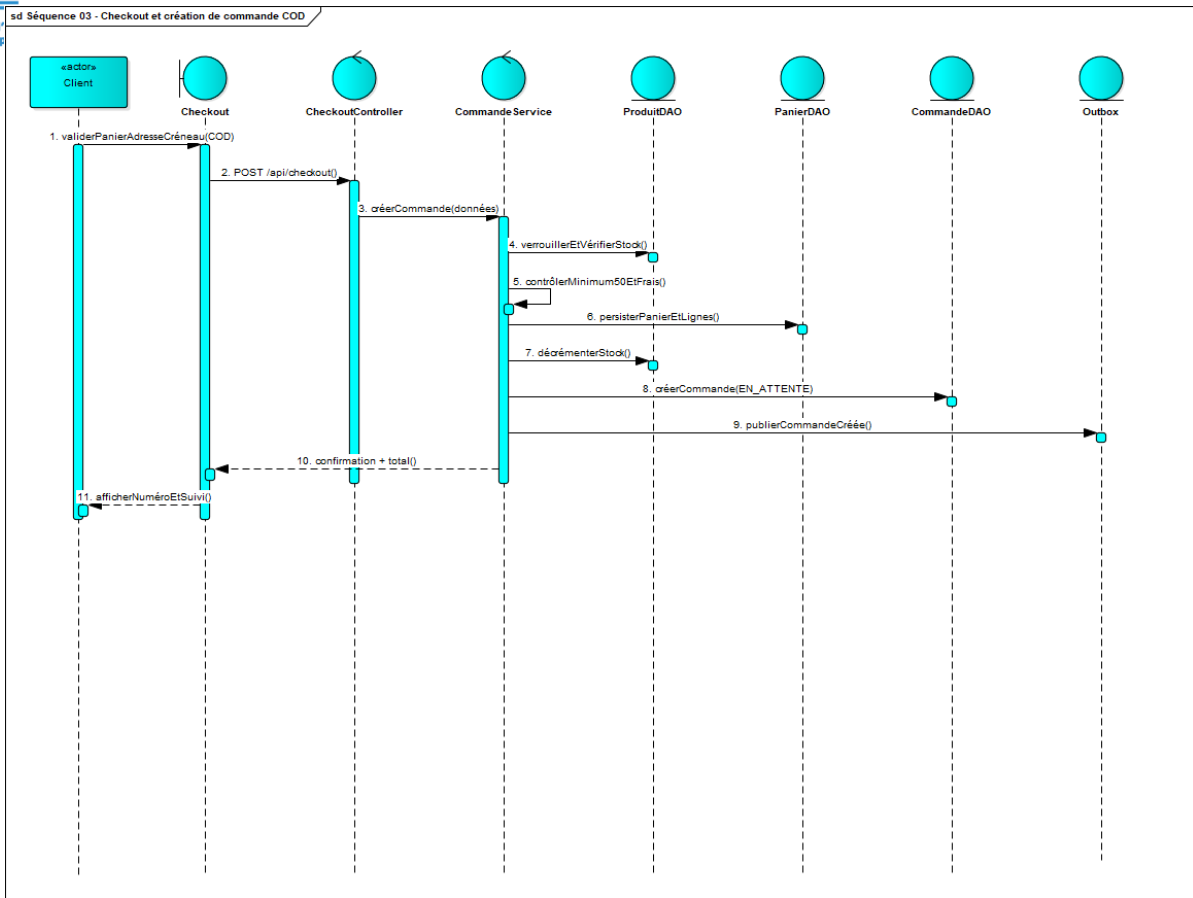


Figure 23 : Séquence complète — Checkout et commande COD

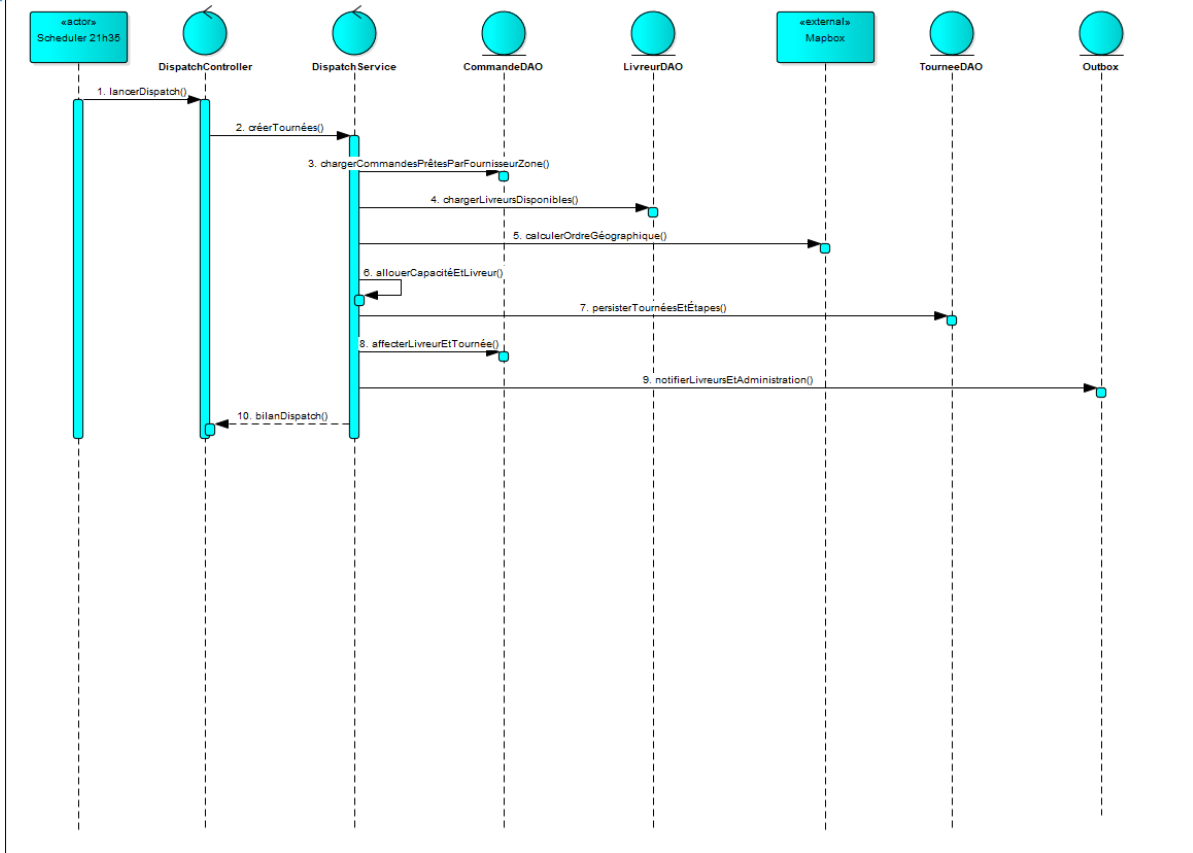


Figure 24 : Séquence complète — Dispatch quotidien

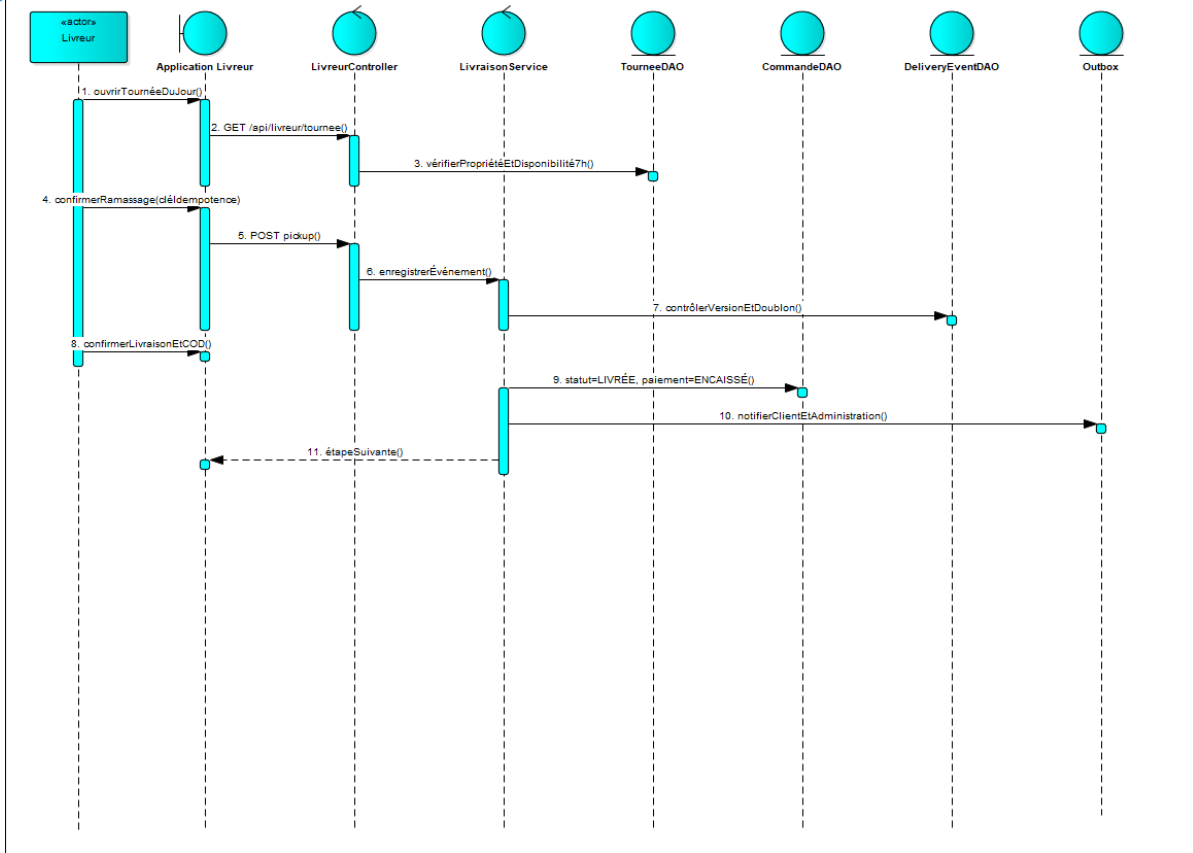
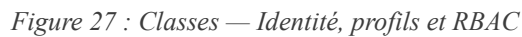
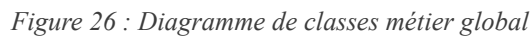


Figure 25 : Séquence complète — Livraison et encaissement COD

Vues de classes



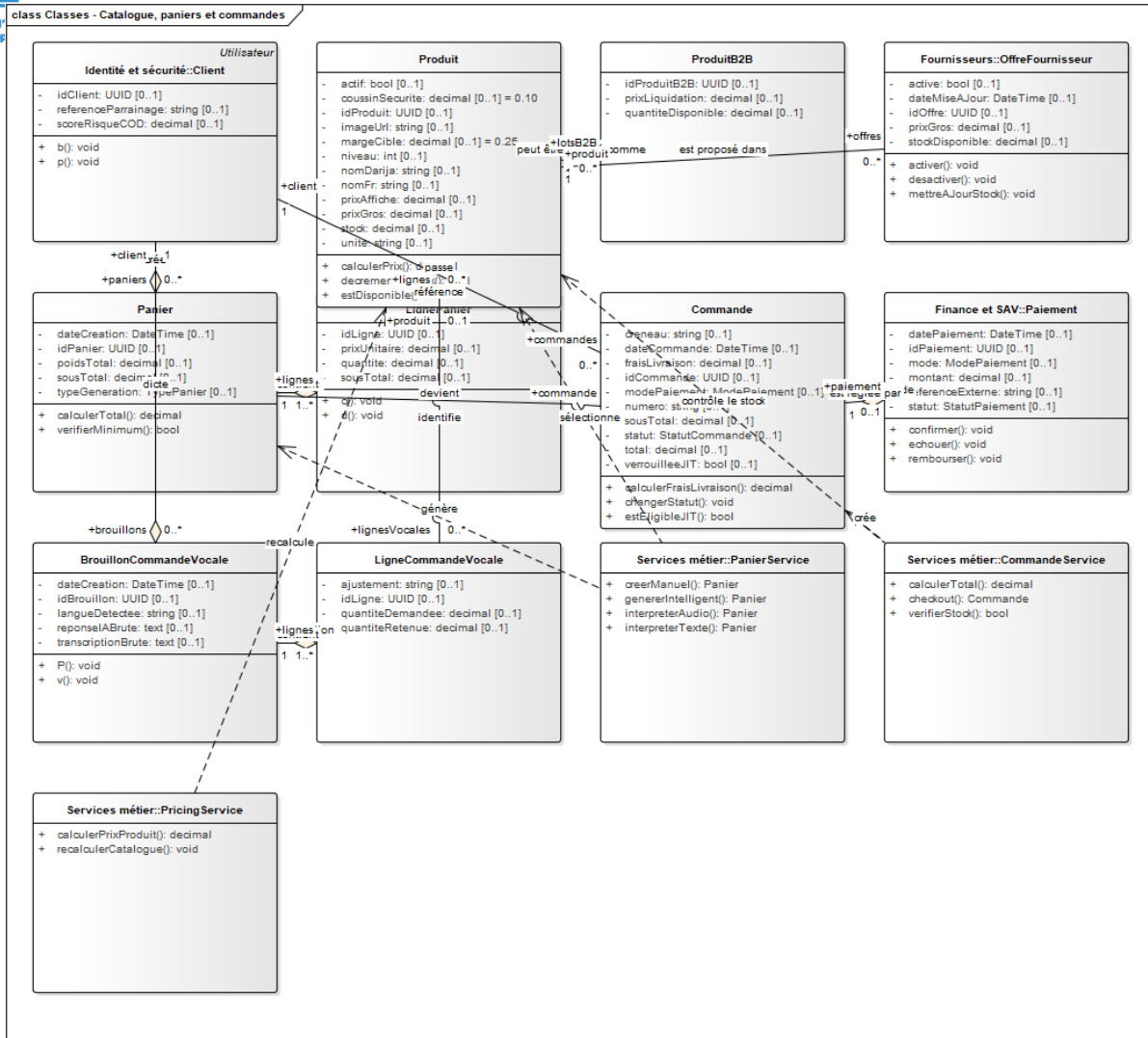


Figure 28 : Classes — Catalogue, paniers et commandes

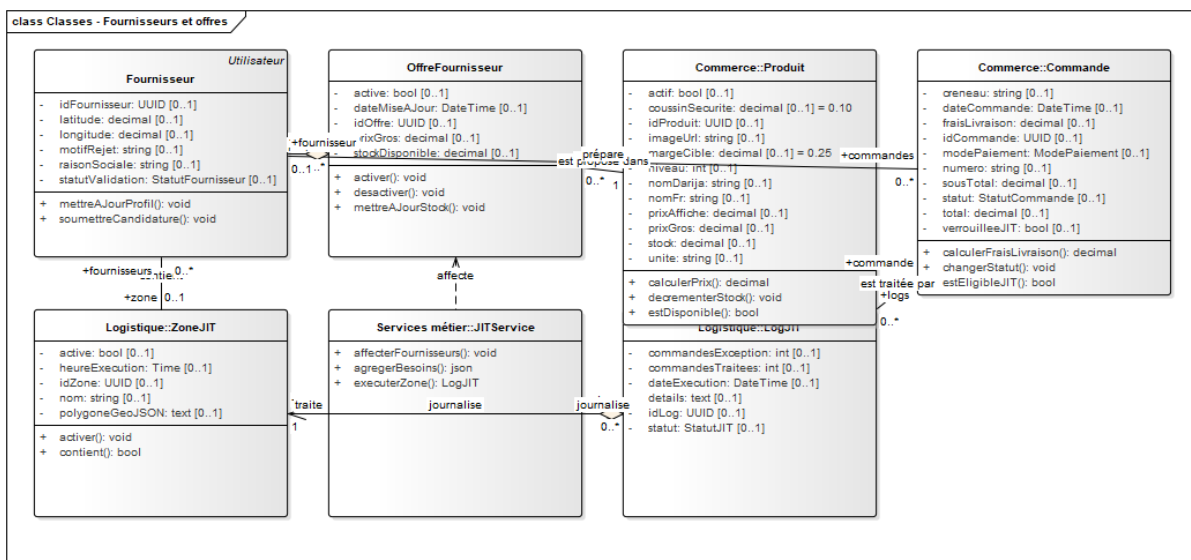


Figure 29 : Classes — Fournisseurs et offres

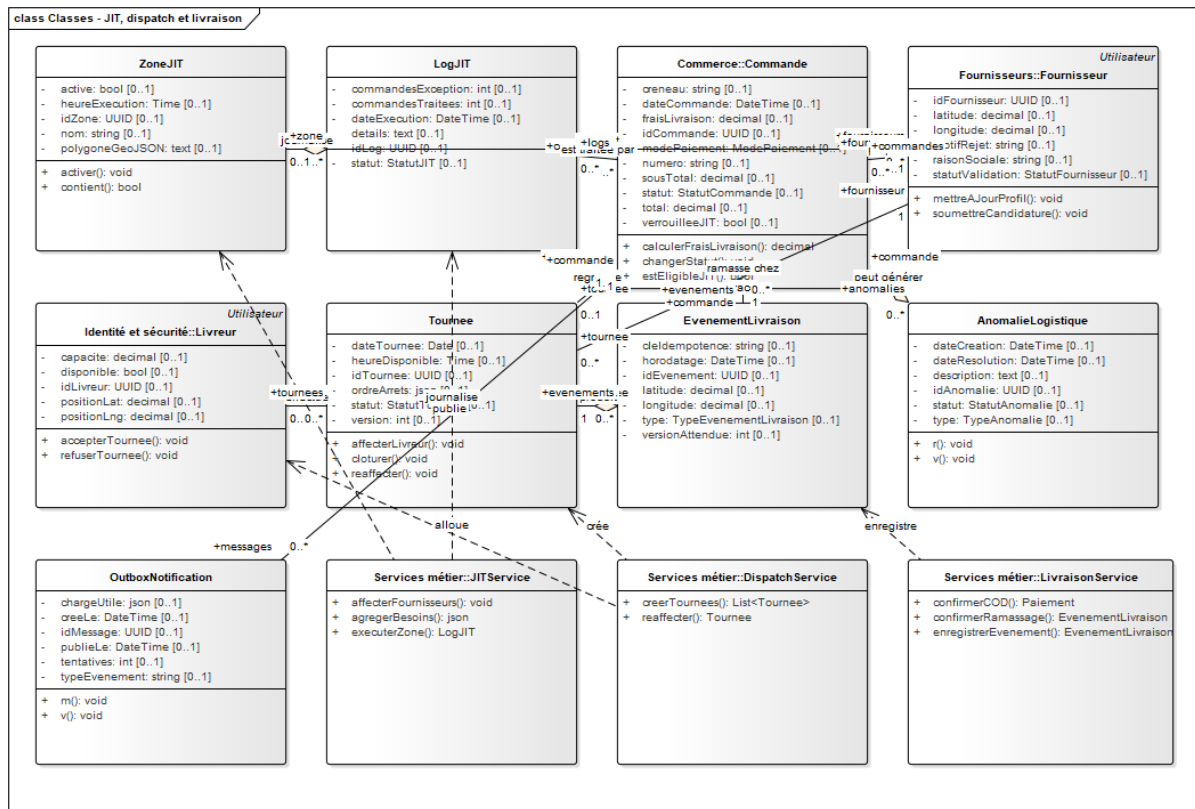


Figure 30 : Classes — JIT, dispatch et livraison

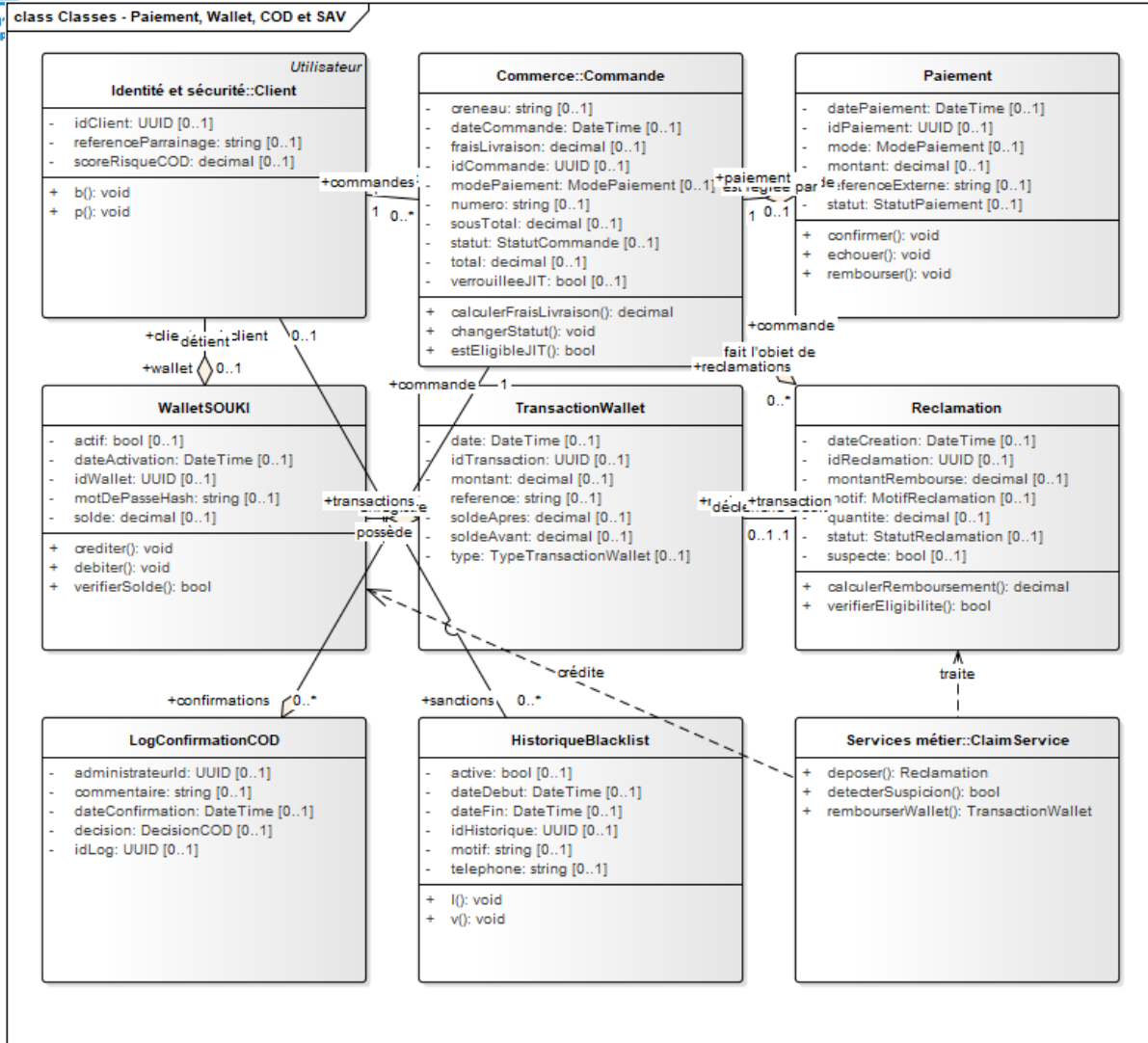
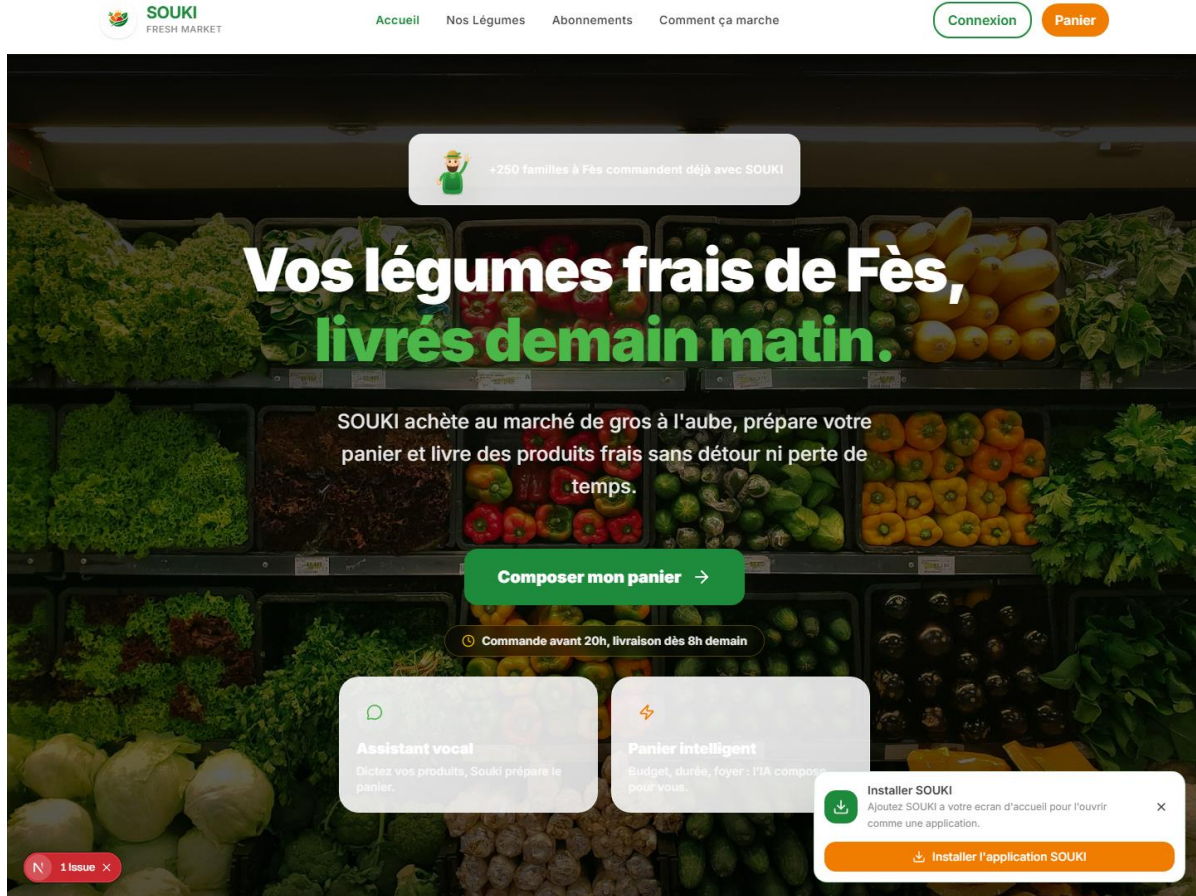


Figure 31 : Classes — Paiement, Wallet, COD et SAV



Page d'accueil — capture réelle de localhost:3000

CATEGORIES

Tous

Légumes

Fruits

Herbes

Commandes ouvertes

Livraison demain pour garantir la fraîcheur.

Connectez-vous pour ajouter des produits au panier et valider votre commande.

Se connecter

1 issue

CATALOGUE DU JOUR

Guide d'achat actif

Ach heb lkhatat ?

Salam, je t'aide a composer un panier frais sans perdre de temps

Choisis toi-même tes produits, parle à l'assistant vocal, ou laisse Souki composer un panier malin selon ton budget.

RECOLTE DE SOUKI

Salam, je peux te composer un panier frais en moins d'une minute.

Commander moi-même

Parcours le catalogue et choisis tes produits librement.

Panier Intelligent

Budget, durée et foyer : Souki compose pour toi.

Assistant IA

Dis tes produits à voix haute, on prépare le panier.

28 produits affichés

Pertinence

Essentiels

Les indispensables du quotidien

2 produits

Votre panier

Le panier est réservé aux utilisateurs connectés.

Connectez-vous pour utiliser le panier, modifier les quantités et commander.

Aller à la connexion

Installer SOUKI

Ajoutez SOUKI à votre écran d'accueil pour l'ouvrir comme une application.

Installer l'application SOUKI

Catalogue — capture réelle de localhost:3000/catalogue



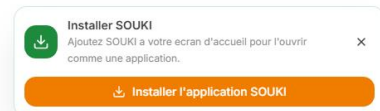
Connexion

Sélectionnez votre type de profil pour continuer.

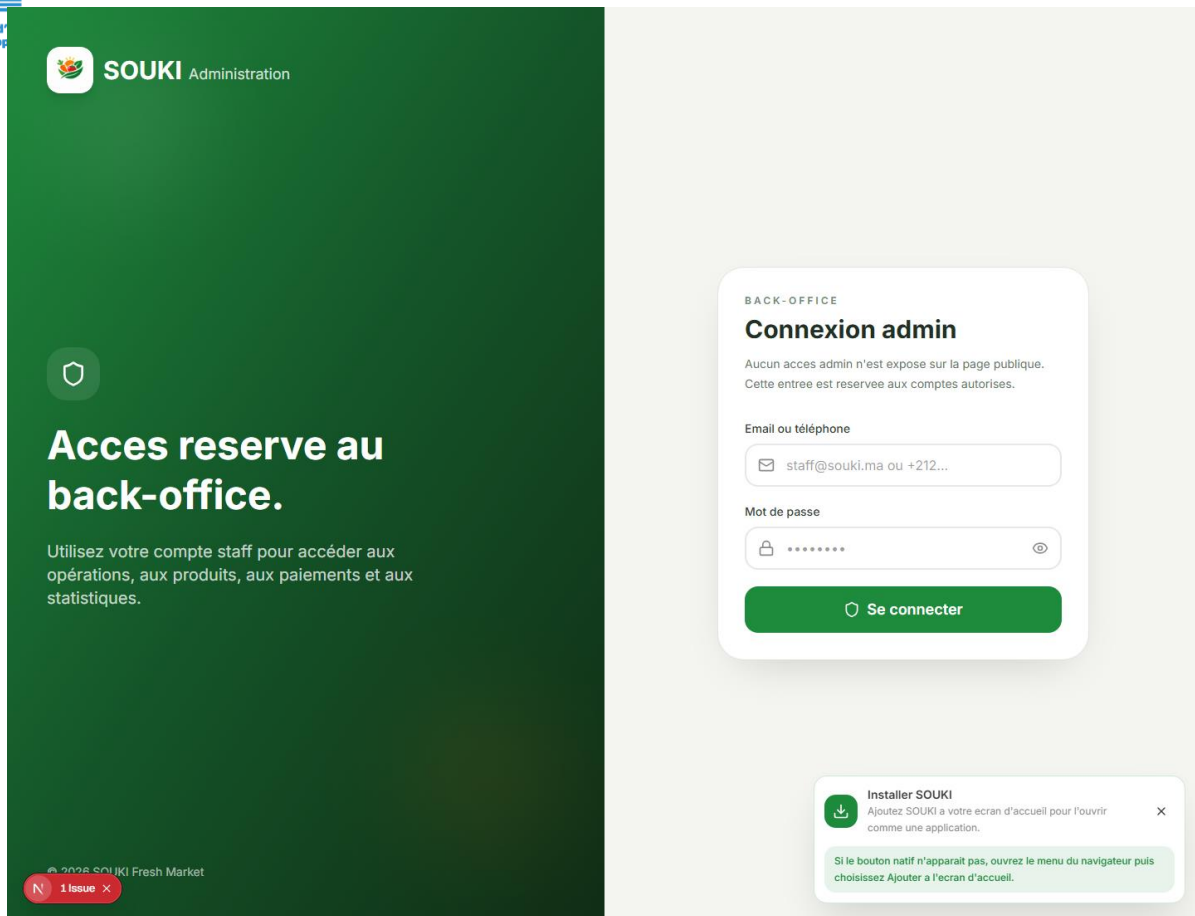
Client
 Pour commander vos produits frais

Parent
 Accédez à l'espace parent

Livreur
 Gérez vos courses et livraisons



Sélection du profil — capture réelle de localhost:3000/login



Connexion administrateur — capture réelle de localhost:3000/admin/login

C. Documentation API REST

La documentation OpenAPI est exposée par FastAPI sur /docs lorsque le backend est démarré.

D. Guide d'installation et déploiement

Installer les dépendances backend, frontend et mobile ; configurer les variables d'environnement ; démarrer PostgreSQL/Supabase, FastAPI et Next.js. Les fichiers README.md et QUICKSTART.md détaillent le lancement.

E. Business Plan détaillé

Les simulations du document pricing_strategy doivent être recalculées avec les données du pilote.